

30-DAY AI CAREER ACCELERATOR

A Hands-On Program for Engineering Students

Designed for: Jathin

3rd Year Engineering · Career Track

Curated by

Prateek Kumar Tiwari

Co-founder, Pinnakle Media · @howprateek

Welcome, Jathin

This document is your complete roadmap for the next 30 days. It's been designed specifically for someone in your position — a 3rd year engineering student who wants to use AI as a career accelerator, not just as a side curiosity. By the end of this program, you won't just "know AI" — you'll have:

- 8+ deployed AI projects on a polished GitHub profile
- A live portfolio site showcasing your work
- A resume that stands out for AI/ML roles and AI-augmented developer roles
- Hands-on experience with the exact stack companies hire for in 2026 (LLM APIs, RAG, vector DBs, agents, prompt engineering)
- A LinkedIn presence that proves you can build, not just talk
- Confidence to clear AI-focused interviews and internship rounds

How each day is structured

Six sections so you never get stuck wondering what to do:

- Focus — the one-line point of the day
- What you'll learn — the concepts you'll have by end of day
- Resources — specific links, in the order to consume them, with time estimates
- Step-by-step tasks — numbered, do them in order
- Tools — what you'll use
- Done when — a checklist to verify you actually finished, not just kind of finished

The one rule: Build in Public

Every week, you'll post your progress on LinkedIn. This is non-negotiable. "Build in public" is how unknown students get noticed by recruiters and founders. By Day 30, you'll have 4–6 LinkedIn posts showing consistent, visible learning — that alone puts you ahead of 95% of engineering students. Templates for every post are in Appendix B at the back of this doc.

Before you ask me a question

Read the Appendix first. It has README templates (Appendix A), LinkedIn post templates (B), code boilerplate for the patterns you'll use every day (C), and a troubleshooting guide for the exact errors you'll hit (D). 90% of small issues are answered in there. If you're stuck for more than 90 minutes AFTER checking the Appendix, then ping me on WhatsApp or Slack. I spend my days building exactly these kinds of AI tools and automations — that's what @howprateek is about. I'd rather unblock you in 10 minutes than have you lose momentum.

What's Inside

Setup & Accounts	Before Day 1 — do this first
Investment Breakdown	What you actually need to spend
Program Overview	Week-by-week summary
Week 1: AI Foundations	Days 1–7
Week 2: Building AI Apps	Days 8–14
Week 3: RAG & Vector Databases	Days 15–21
Week 4: Agents, Capstone & Career	Days 22–30
Capstone Project Ideas	7 ideas to pick from on Day 24
Resume Template for AI Roles	Day 28 reference
Daily Progress Tracker	30-row checklist
Appendix A: README Template	For every project repo
Appendix B: LinkedIn Post Templates	Daily, weekly, launch posts
Appendix C: Code Boilerplate	API calls, Streamlit, env setup
Appendix D: Troubleshooting	Common errors and exact fixes
Appendix E: Glossary	Every jargon term explained
Ground Rules & FAQ	What to do when things go sideways

Before Day 1: Setup & Accounts

Do this on Day 0 — the evening before you start. Takes about 90 minutes. If you don't finish, Day 1 picks up where you leave off.

Hardware

- Laptop with at least 8GB RAM (16GB ideal). Anything 4 years old or newer works.
- Stable internet — most of this is API and cloud work, not heavy local compute.
- No GPU needed. You're going to USE models via APIs, not train them. That's what 99% of AI jobs actually do in 2026.

Accounts to create (all free except where noted)

Service	URL	Why
GitHub	github.com	Code portfolio. Use real name + photo.
Google AI Studio	aistudio.google.com	Free Gemini API access. Very generous limits.
Anthropic Console	console.anthropic.com	Claude API. Needs \$5 credit (see Investment).
Hugging Face	huggingface.co	Deploy AI apps free with 16GB RAM.
Streamlit Cloud	share.streamlit.io	Deploy unlimited public Streamlit apps.
Vercel	vercel.com	Free hosting for portfolio site.
Hashnode	hashnode.com	Free dev blog. Sign up with GitHub.
LinkedIn	linkedin.com	Update photo, headline, About.

Software to install

Software	Download from	Verify with terminal command
Python 3.12	python.org/downloads	python --version (or python3 --version)
VS Code	code.visualstudio.com	Just open it
Cursor	cursor.com	Just open it (2-week Pro trial included)
Git	git-scm.com	git --version
Node.js 20+	nodejs.org	node --version

Investment Breakdown

Honest numbers. Most of this program runs on free tiers. The only essential paid item is a small Claude API credit (about ₹450). Everything else is optional.

Essential (₹450)

Item	Cost	Why
Anthropic API credit	\$5 (~₹450)	Lasts the whole month for learning use.

Recommended (₹1,800–₹3,500)

Pick one or two. Claude Pro is the highest-leverage one since you'll use it dozens of times daily as a tutor.

Item	Cost	Why
Claude Pro (1 month)	\$20 (~₹1,800)	Best AI for learning, coding help, debugging.
Cursor Pro (1 month)	\$20 (~₹1,800)	AI-first code editor. Free 2-week trial first.
OpenAI API credit	\$5 (~₹450)	Optional. Compare Claude vs GPT in projects.
Custom domain (jathin.dev)	~₹800/year	Optional. Makes portfolio look serious.

Free Forever (₹0)

- Google AI Studio / Gemini API — generous free tier
- GitHub, GitHub Pages — free hosting
- Streamlit Cloud — deploy unlimited public apps
- Hugging Face Spaces — deploy AI apps with 16GB RAM
- Vercel — free hobby tier
- ChromaDB — free open-source vector database
- Cursor free tier, VS Code, Python, Node.js, Git — all free

Bottom line

Minimum spend: ₹450 (Anthropic API credit). Sweet spot: ₹2,200–₹2,500 (API + Claude Pro). Premium: ₹5,000 covers everything comfortably.

Program Overview

Four weeks. Each week builds on the previous one. Don't skip ahead — the order matters.

Week	Theme	What You'll Learn	Projects You'll Ship
Week 1	AI Foundations	How LLMs work Prompt engineering Calling AI APIs Python + Git basics	Prompt library AI CLI tool Note summarizer
Week 2	Building AI Apps	Streamlit UI Multi-feature apps Image AI Production deployment	AI Chatbot Content generator OCR + LLM app Resume Reviewer
Week 3	RAG & Vector DBs	Embeddings ChromaDB RAG pipelines LangChain basics	Semantic search Chat with PDF Chat with YouTube
Week 4	Agents + Capstone + Career	AI agents Tool use Resume polish Interview prep	Research Agent Capstone project Portfolio site

Time commitment

- Average 5–6 hours/day on weekdays, 4 hours on weekends
- Some days (capstone build days) will hit 8 hours — plan for that
- If you miss a day, don't double up next day. Just shift forward by one day.

Week 1: AI Foundations

Goal: Understand what LLMs are, how to talk to them well, and build your first AI tool. By end of Week 1 you'll have made your first API call and shipped your first AI app to a GitHub repo.

DAY 1 · Orientation & Public Commitment (4-5 hours)

Focus: Set up your environment, create all accounts, and announce the journey publicly so you can't quit silently.

What you'll learn today:

- How GitHub works (repos, README, commits, .gitignore)
- How to set up a clean dev environment on your laptop
- The 'build in public' mindset and why it matters for jobs
- Basic terminal navigation

Resources (use in this order):

- **Watch** (15 min): Git & GitHub for Beginners by Fireship → youtube.com/watch?v=HkdAHXoRtos
- **Read** (10 min): GitHub Hello World → docs.github.com/en/get-started/start-your-journey/hello-world
- **Read** (5 min): What is 'build in public' → buildinpublic.com (skim the homepage)
- **Reference** (as needed): Python install guide → realpython.com/installing-python/

Step-by-step tasks:

1. Install Python 3.12 from python.org. Verify in terminal: `python --version` (Mac/Linux may need `python3 --version`). Should show 3.12.x
2. Install Git from git-scm.com. Verify: `git --version` shows 2.x or higher
3. Install Node.js 20+ from nodejs.org. Verify: `node --version` shows v20+
4. Install VS Code from code.visualstudio.com (just download and open it)
5. Install Cursor from cursor.com (free tier is fine; Pro is 2-week trial)
6. Configure git globally: `git config --global user.name "Jathin"` and `git config --global user.email "your-email@gmail.com"`
7. Create accounts at: github.com, aistudio.google.com (get free API key), console.anthropic.com (we'll add credit on Day 5), huggingface.co, share.streamlit.io, vercel.com, hashnode.com
8. On GitHub: click 'New repository' → name it 'ai-30-day-journey' → Public → check 'Add a README file' + 'Add .gitignore' (choose Python) + 'Choose a license' (MIT) → Create repository
9. Clone it locally. Open terminal, navigate to where you keep code (e.g., `cd ~/Documents`), then: `git clone https://github.com/YOUR_USERNAME/ai-30-day-journey`
10. Open the cloned folder in VS Code: `cd ai-30-day-journey && code .`
11. Edit README.md using the README template from Appendix A. Fill in your name, what you'll do for 30 days, and a link to your LinkedIn (leave it as TODO if not posted yet).
12. Commit your changes: `git add . && git commit -m "Day 1: Starting the 30-day AI journey" && git push`
13. Verify the change on GitHub by refreshing your repo page in browser
14. Update LinkedIn headline to: '3rd Year CS @ [Your College] | Building AI Apps | Python, LLMs, RAG'
15. Write LinkedIn announcement post using template from Appendix B (Day 1 template). Include the GitHub repo link.

Tools you'll use: Terminal, Python, Git, VS Code, GitHub, LinkedIn

✓ Done when:

- ☐ All 5 software tools verified working from terminal (python, git, node, code, cursor)

- ☐ All 7 accounts created and you can log into each
- ☐ Repo 'ai-30-day-journey' exists on GitHub and shows your first commit
- ☐ README has your name, 30-day commitment statement, and (placeholder for) LinkedIn post link
- ☐ LinkedIn post is live with #BuildInPublic and #AIEngineer tags
- ☐ LinkedIn headline updated

DAY 2 · Python Refresher + Git Daily Flow (5 hours)

Focus: Sharpen Python basics and master the Git workflow you'll use every day for the next 28 days.

What you'll learn today:

- Python data types, list/dict comprehensions, functions, classes, error handling
- Virtual environments (why they exist and how to use them)
- The Git daily flow: add → commit → push → pull → branch → merge
- The .env file pattern for storing API keys safely

Resources (use in this order):

- **Watch** (1 hr): Python in 1 Hour by Mosh → youtube.com/watch?v=kqtD5dpn9C8
- **Practice** (1 hr): HackerRank Python (Introduction track, 5 easy problems) → hackerrank.com/domains/python
- **Watch** (30 min): Git Branching Tutorial by Fireship → youtube.com/watch?v=e2lbNHi4uCI
- **Read** (15 min): python-dotenv documentation → pypi.org/project/python-dotenv/
- **Reference** (as needed): Virtual environments primer → realpython.com/python-virtual-environments-a-primer/

Step-by-step tasks:

1. Open terminal. Create new folder: `mkdir python-warmup && cd python-warmup`
2. Create a virtual environment: `python -m venv venv` (or `python3 -m venv venv` on Mac)
3. Activate it. Windows: `venv\Scripts\activate`. Mac/Linux: `source venv/bin/activate`. You should see (venv) appear in your terminal prompt.
4. Install dotenv: `pip install python-dotenv`
5. Create a .env file with: `MY_NAME=Jathin` and `DUMMY_KEY=test123` on two lines
6. Create `read_env.py` using the snippet in Appendix C.1. Run it: `python read_env.py`. It should print your name and dummy key.
7. Create .gitignore file (in same folder). Add two lines: `venv/` and `.env` (this is critical — NEVER commit .env to GitHub)
8. Go to hackerrank.com, sign in with GitHub, navigate to Python domain. Solve 5 easy problems from 'Introduction' and 'Basic Data Types'.
9. Save each solution as `problem1.py`, `problem2.py`, ... `problem5.py` in the python-warmup folder
10. Initialize git: `git init`
11. Check what git sees: `git status`. The `venv/` folder and `.env` should be hidden (because of .gitignore). If you see them, fix .gitignore first.
12. First commit: `git add . && git commit -m "Day 2: Python warmup with 5 HackerRank solutions"`
13. Create a new GitHub repo called 'python-warmup' (Public, do NOT add README, .gitignore, or license this time — we already have them)
14. Connect and push: `git remote add origin https://github.com/YOUR_USERNAME/python-warmup.git && git branch -M main && git push -u origin main`
15. Verify on github.com that .env and venv/ are NOT shown in the repo (very important — if they

ARE there, see Appendix D for fix)

16. Practice branching: `git checkout -b feature/notes` → create a small file `notes.md` with anything → `git add .` && `git commit -m "branch test"` → `git checkout main` → `git merge feature/notes` → `git push`

Tools you'll use: Python, pip, venv, Git, GitHub, HackerRank

✓ **Done when:**

- ☐ You can activate/deactivate a venv without checking notes
- ☐ 'python-warmup' repo exists on GitHub with 5 solved problems and the dotenv demo
- ☐ `.env` and `venv/` are NOT visible on the GitHub repo page (verify in browser)
- ☐ You've completed at least one branch + merge cycle locally
- ☐ GitHub contribution graph shows Day 2 commits (green square on today's date)

DAY 3 · How LLMs Actually Work (4 hours)

Focus: *Build the mental model that will let you reason about every AI tool you use after this.*

What you'll learn today:

- What tokens are and why they matter
- Context window (and what happens when you exceed it)
- Temperature, top-p, and how they affect output
- System prompts vs user messages
- What an LLM can and can't do (very important for setting expectations)

Resources (use in this order):

- **Watch** (40 min): 3Blue1Brown: But what is a GPT? → youtube.com/watch?v=wjZofJX0v4M
- **Read** (20 min): Anthropic Introduction to Claude → docs.claude.com/en/docs/intro
- **Try** (15 min): OpenAI Tokenizer (paste different texts) → platform.openai.com/tokenizer
- **Watch** (1 hr, optional): Karpathy: Intro to Large Language Models → youtube.com/watch?v=zjkBMFhNj_g
- **Reference** (as needed): Anthropic glossary → docs.claude.com/en/docs/about-claude/glossary

Step-by-step tasks:

1. Watch the 3Blue1Brown video. Take notes on three concepts: token, context window, attention. One sentence each in your own words.
2. Go to platform.openai.com/tokenizer. Paste three different texts: an English sentence, a Hindi/Kannada sentence (in Devanagari/Kannada script), and a Python code snippet. Note how the token count differs.
3. Open Claude.ai (free tier is fine). Ask: 'Explain context window in 2 sentences.' Then ask: 'What is the context window for Claude Sonnet 4?' Note both answers.
4. Go to aistudio.google.com. Set temperature to 0.0 and ask 'Write a one-sentence story about a cat.' Repeat 3 times — notice it's the same or very similar.
5. Now set temperature to 1.0. Ask the same question 3 times. Notice how varied the outputs become.
6. Set a system prompt in AI Studio: 'You are a sarcastic teenager.' Ask 'What is photosynthesis?' Note how tone changes.
7. Open VS Code in your 'ai-30-day-journey' folder. Create a new file: `llm-notes.md`
8. Write a 200-word explanation in your own words covering: (a) what an LLM is, (b) what tokens are, (c) what context window means, (d) what temperature does, (e) what a system prompt is.
9. Commit and push: `git add llm-notes.md` && `git commit -m "Day 3: LLM fundamentals notes"` && `git push`

10. Optional but recommended: Create a 5-slide LinkedIn carousel in Canva using your notes. Title: 'LLMs explained in 5 slides for non-engineers.' Post it.

Tools you'll use: YouTube, Claude.ai, Google AI Studio, OpenAI Tokenizer, VS Code

✓ **Done when:**

- ☐ llm-notes.md exists in your repo with at least 5 concepts in your own words
- ☐ You can answer (without looking up) what a token is, what context window is, what temperature does
- ☐ You've used Claude.ai and AI Studio for at least 30 minutes each, comparing outputs
- ☐ You've experimented with temperature 0 vs 1.0 and can describe the difference

DAY 4 · Prompt Engineering Masterclass (5 hours)

Focus: *The highest-leverage skill in this whole program. Practice deliberately, not casually.*

What you'll learn today:

- Zero-shot vs few-shot prompting (and when to use each)
- Chain-of-thought reasoning
- XML tags for prompt structure (Anthropic's signature technique)
- Role / persona prompting
- Prefill technique for guaranteed output format

Resources (use in this order):

- **Read** (30 min): Anthropic Prompt Engineering Overview → docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- **Read** (20 min): Be clear and direct → docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct
- **Read** (20 min): Use examples (few-shot) → docs.claude.com/en/docs/build-with-claude/prompt-engineering/multishot-prompting
- **Read** (20 min): Chain of thought → docs.claude.com/en/docs/build-with-claude/prompt-engineering/chain-of-thought
- **Read** (20 min): Use XML tags → docs.claude.com/en/docs/build-with-claude/prompt-engineering/use-xml-tags
- **Reference** (deep-dive): Prompt Engineering Guide → promptingguide.ai

Step-by-step tasks:

1. Open Claude.ai. Try this exact prompt: 'Write a poem.' Note how generic the output is.
2. Now try a structured prompt: 'Write a 4-line poem about Bangalore traffic in the style of Rumi. Use sensory imagery (sounds, smells, chai). Make it bittersweet.' Compare.
3. Practice zero-shot: Paste 5 product reviews and ask Claude to classify each as Positive, Negative, or Neutral. Note accuracy.
4. Practice few-shot: This time give Claude 3 example classifications first (review → label), then ask it to classify 5 new ones. Compare to zero-shot.
5. Practice CoT: Ask Claude '17 × 23 = ?' then ask the same with 'Show your work step by step.' Notice the difference in accuracy.
6. Practice XML tags: Ask Claude to summarize an article. Wrap the article in <article>...</article> tags and the task in <task>...</task> tags. See how much cleaner this is than running text.
7. Practice role prompting: Same question 'What should I name my startup?' asked once normally, once with 'You are Paul Graham. Answer as he would.' Compare.
8. Create a new GitHub repo: 'prompt-library' (Public, with README)
9. Clone it locally, open in VS Code

10. Create 5 folders inside: customer-support, content-creation, coding-assistance, data-analysis, personal-productivity
11. Write 4 prompts in each folder (20 total) as .md files. Each file uses the template from Appendix C.2 (purpose / prompt / example input / example output)
12. Update README.md with a table of contents linking to each prompt
13. Commit and push: `git add . && git commit -m "Day 4: Prompt library with 20 prompts" && git push`

Tools you'll use: Claude.ai, Google AI Studio, VS Code, GitHub

✓ **Done when:**

- ☐ 'prompt-library' repo exists on GitHub with 20 prompts across 5 folders
- ☐ Each prompt file follows the Appendix C.2 template (purpose, prompt, example I/O)
- ☐ README has a table of contents with working links to each prompt file
- ☐ You can explain (without looking) the difference between zero-shot, few-shot, and chain-of-thought
- ☐ You've used XML tags in at least 5 of your library prompts

DAY 5 · Your First API Call (5 hours)

Focus: *The leap from 'using AI' to 'writing software that uses AI'.*

What you'll learn today:

- REST APIs basics (request, response, headers, auth)
- Using an SDK vs raw HTTP calls
- API key security (.env files, environment variables)
- Streaming responses
- Basic error handling

Resources (use in this order):

- **Read** (20 min): Anthropic Quickstart → docs.claude.com/en/api/getting-started
- **Read** (15 min): Anthropic Python SDK README → github.com/anthropics/anthropic-sdk-python
- **Read** (20 min): Gemini Python quickstart → ai.google.dev/gemini-api/docs/quickstart?lang=python
- **Read** (15 min): Streaming docs → docs.claude.com/en/api/streaming
- **Reference** (as needed): Python argparse tutorial → docs.python.org/3/howto/argparse.html

Step-by-step tasks:

1. Go to console.anthropic.com → Settings → Billing → Add \$5 credit (this is the only essential paid item in the program)
2. Console → API Keys → Create Key. Copy it (starts with sk-ant-). Save in a notes file — you can't see it again after closing.
3. Go to aistudio.google.com → 'Get API key' → 'Create API key'. Copy it. It's free.
4. Create new project: `mkdir ask-ai && cd ask-ai`. Setup venv (same way as Day 2) and activate it.
5. Install dependencies: `pip install anthropic google-generativeai python-dotenv click`
6. Create .env file with two lines: `ANTHROPIC_API_KEY=sk-ant-YOUR-KEY-HERE` and `GEMINI_API_KEY=YOUR-GEMINI-KEY-HERE`
7. Create .gitignore with: `venv/` and `.env`
8. Create `hello_claude.py` using the boilerplate from Appendix C.3. Run it: `python hello_claude.py`. You should see Claude reply.
9. Create `hello_gemini.py` using Appendix C.4. Run it. You should see Gemini reply.
10. Now build the real CLI. Create `ask.py` using Appendix C.5 boilerplate as starting point. It should

accept: python ask.py "your question"

11. Add a `--model` flag: python ask.py "What is RAG?" `--model claude` OR `--model gemini`. Default to claude.
12. Add `--stream` flag for streaming output (tokens print as they arrive — see Appendix C.6 for streaming code)
13. Add error handling: missing API key → friendly message, empty question → friendly message, API error → catch and print clearly (not full traceback)
14. Test all four flows: ask.py with claude, with gemini, with `--stream`, with empty question
15. Write README using Appendix A template. Include 3 usage examples with screenshots of the terminal output.
16. Create GitHub repo 'ask-ai' (Public, no auto README). Push your code.

Tools you'll use: Anthropic API, Gemini API, Python, Click library, GitHub

✓ **Done when:**

- ☐ 'ask-ai' repo exists on GitHub
- ☐ Running python ask.py "Hello" returns a response from Claude
- ☐ Running python ask.py "Hello" `--model gemini` returns a response from Gemini
- ☐ Running python ask.py "Tell me a story" `--stream` streams the response token by token
- ☐ README has 3 usage examples with screenshots
- ☐ .env is NOT visible in the GitHub repo (check on github.com/YOUR/ask-ai)
- ☐ Your \$5 Anthropic credit shows usage on console.anthropic.com (should still have most of it left)

DAY 6 · Mini Project: AI Note Summarizer (6 hours)

Focus: First real, useful tool. Something you'll actually use yourself for college lectures and YouTube videos.

What you'll learn today:

- File I/O in Python (reading text files)
- Working with YouTube transcripts programmatically
- Structured prompts (asking for specific output format)
- Writing a clean CLI with multiple input types

Resources (use in this order):

- **Read** (15 min): youtube-transcript-api docs → pypi.org/project/youtube-transcript-api/
- **Read** (15 min): Python file handling → realpython.com/read-write-files-python/
- **Read** (15 min): Anthropic: Increase output consistency → docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails/increase-consistency
- **Watch** (5 min): Loom tutorial for screen recording (any short YouTube video) → youtube.com/results?search_query=loom+tutorial+beginner

Step-by-step tasks:

1. New folder: `mkdir note-summarizer && cd note-summarizer`. Setup venv, .env (copy from ask-ai), .gitignore.
2. Install: `pip install anthropic youtube-transcript-api python-dotenv click`
3. Create summarize.py. CLI signature: `python summarize.py <file-or-youtube-url>`
4. Logic: detect if input starts with 'https://' and contains 'youtube' or 'youtu.be' → treat as YouTube URL. Otherwise → treat as file path.
5. If YouTube: extract video ID (the `v=` parameter or last part of `youtu.be/` URL). Use `youtube_transcript_api.YouTubeTranscriptApi.get_transcript(video_id)` to fetch transcript.

Concatenate all 'text' fields.

6. If file: open and read content with: `with open(path) as f: content = f.read()`
7. Build a prompt that asks Claude to return three sections: (1) 3-sentence summary, (2) 5 key points as bullets, (3) 3 action items. Wrap the input in `<transcript>` or `<document>` XML tags.
8. Use Anthropic 'prefill' technique: end your prompt with `'<summary>'` so Claude starts there. This guarantees format.
9. Parse Claude's response. Print to console with nice formatting (headers in bold or with `===` separators).
10. Add `--output file.md` flag: if provided, save summary as a markdown file instead of just printing.
11. Handle these errors gracefully (don't show stack traces, show friendly messages): invalid URL, no transcript available (some videos have none), file not found, API errors, empty input.
12. Test 1: Create a file `test.txt` with any article. Run: `python summarize.py test.txt`
13. Test 2: Pick a 10-minute YouTube video you've watched. Run: `python summarize.py "https://www.youtube.com/watch?v=..."`
14. Test 3: Try an invalid URL and a missing file to verify error handling works
15. Write README using Appendix A template. Add 2 screenshots: one of CLI summarizing a text file, one of a YouTube video.
16. Record a 60-second demo: Loom (free at loom.com) is easiest. Show running the command and the output appearing. Save the link.
17. Embed Loom link in your README.
18. Create GitHub repo 'note-summarizer', push everything.

Tools you'll use: Python, Anthropic API, youtube-transcript-api, Loom

✓ Done when:

- ☐ `python summarize.py notes.txt` produces a clean 3-section summary
- ☐ `python summarize.py "https://youtube.com/..."` works on a real YouTube video
- ☐ Error handling tested with 3 bad inputs (no crashes, friendly messages)
- ☐ README has 2 screenshots and a Loom demo link
- ☐ 'note-summarizer' repo is public on GitHub

DAY 7 · Week 1 Recap + Personal Blog Launch (4 hours)

Focus: Consolidate the week. Build the public-facing learning hub you'll keep updating.

What you'll learn today:

- Writing technical blog posts (structure, tone, code formatting)
- Using Hashnode for free dev blogging
- Creating LinkedIn carousels that actually get engagement
- Cross-promoting content across platforms

Resources (use in this order):

- **Sign up** (5 min): Hashnode (use GitHub login) → hashnode.com
- **Read** (10 min): How to start a dev blog (Hashnode's own guide) → hashnode.com/start-a-blog
- **Reference** (browse): Canva LinkedIn carousel templates → canva.com/templates/?query=linkedin+carousel
- **Reference** (browse for inspiration): Top AI builder LinkedIn posts → linkedin.com/search/results/content/?keywords=%23BuildInPublic

Step-by-step tasks:

1. Sign up at hashnode.com with GitHub. Choose blog URL: jathin.hashnode.dev (or similar). Pick a

clean theme.

2. Add a blog bio: '3rd year engineering student. Documenting my 30-day AI journey. Building in public.'
3. Click 'New post'. Title: 'Week 1 of my 30-day AI challenge: 5 things I learned about LLMs'
4. Structure: (1) intro paragraph — who you are, what you're doing. (2) 5 numbered lessons, each 2-3 paragraphs. (3) What you built (list 4 GitHub repos with links). (4) What's next. (5) Closing CTA — 'Follow on LinkedIn for updates'
5. Add screenshots of your Day 6 note-summarizer in action. Use the same screenshots from your README.
6. Add code blocks for one or two interesting bits of code (use Hashnode's code block feature)
7. Publish the post. Copy the URL.
8. Now create the LinkedIn carousel. Open Canva → search 'LinkedIn carousel' → pick a clean template
9. Slides (8-10 total): (1) Hook slide: 'I built 4 AI tools in 7 days as a 3rd year student. Here's how →' (2-5) One slide per project: name, what it does, screenshot, tech used (6) The biggest lesson you learned (7) The mistake you made and fixed (8) What's coming next week (9) Final CTA: 'Follow me for Week 2 updates'
10. Download as PDF or images. Upload as carousel to LinkedIn (use 'Add document' option for cleanest carousel)
11. Caption: use the LinkedIn template from Appendix B (Week 1 recap version)
12. Post on LinkedIn. Tag: #BuildInPublic #AIEngineer #100DaysOfCode #LearningAI #IndianTech
13. In the FIRST comment of your own post, drop the link to your Hashnode blog post (LinkedIn boosts posts with low first-comment-link engagement)
14. Reply to every comment within 24 hours

Tools you'll use: Hashnode, Canva, LinkedIn

✓ **Done when:**

- ☐ Hashnode blog is live at your URL with the Week 1 recap post
- ☐ LinkedIn carousel published with 8+ slides
- ☐ Blog post link is in the first comment of the LinkedIn post
- ☐ At least 10 engagements (likes/comments) on the LinkedIn post within 24 hours
- ☐ You've replied to every comment received

Week 2: Building AI Apps

Goal: Move from CLI scripts to deployed web apps. By end of Week 2 you'll have 4 live, shareable URLs of AI products you built — this is what makes recruiters reach out.

DAY 8 · Streamlit Crash Course (5 hours)

Focus: *Learn the tool that lets you ship a polished web UI in 30 minutes.*

What you'll learn today:

- Streamlit core widgets (text_input, file_uploader, button, chat_input)
- Streamlit layout (columns, sidebar, expanders, tabs)
- Session state for persistent data across reruns
- Deploying to Streamlit Cloud

Resources (use in this order):

- **Go through** (1.5 hr): Streamlit Get Started → docs.streamlit.io/get-started
- **Try** (1 hr): 30 Days of Streamlit (do days 1-7 only) → 30days.streamlit.app
- **Browse** (20 min for inspiration): Streamlit App Gallery → streamlit.io/gallery
- **Read** (20 min): Session state concepts → docs.streamlit.io/develop/concepts/architecture/session-state
- **Reference** (as needed): Streamlit deployment guide → docs.streamlit.io/deploy/streamlit-community-cloud

Step-by-step tasks:

1. New folder: `mkdir streamlit-practice && cd streamlit-practice`. Setup venv and activate.
2. Install: `pip install streamlit`
3. Create `hello.py` with the Streamlit hello world from Appendix C.7
4. Run it: `streamlit run hello.py`. It should open in your browser at `localhost:8501`
5. Practice app 1: `form_demo.py` — uses `text_input`, `number_input`, `selectbox`, `button`. When button clicked, display the values.
6. Practice app 2: `upload_demo.py` — `file_uploader` for an image. Display the uploaded image.
7. Practice app 3: `chat_demo.py` — `st.chat_message` and `st.chat_input`. Add `session_state` to remember messages across reruns. (See Appendix C.8)
8. Use Appendix C.9 to add a sidebar with controls.
9. Now deploy `hello.py` to the cloud:
10. Create a new GitHub repo: 'streamlit-hello'. Add `hello.py` and a `requirements.txt` (just one line: `streamlit`). Push.
11. Go to `share.streamlit.io`. Click 'New app'. Select the repo, branch (main), file (`hello.py`). Click Deploy.
12. Wait 2-3 minutes for build. Get your URL. Should look like: `streamlit-hello-yourusername.streamlit.app`
13. Open the URL in browser. Verify it works.
14. Commit your practice apps too: `git add . && git commit -m "Day 8: Streamlit practice" && git push`

Tools you'll use: Streamlit, Streamlit Cloud, GitHub

✓ Done when:

- ☐ All 3 practice apps run locally without errors (form, upload, chat)
- ☐ Your first Streamlit app is live on streamlit.io URL
- ☐ You understand `session_state` — can explain why `st.session_state` is needed for chat history

- ☐ GitHub repo 'streamlit-hello' is deployed and visible to anyone with the URL

DAY 9 · AI Chatbot with Memory (6 hours)

Focus: *Your own ChatGPT clone — but yours, deployed, on your resume.*

What you'll learn today:

- Building a proper chat interface in Streamlit
- Conversation memory with session_state
- Sending conversation history to the API correctly
- Streaming responses in Streamlit
- Custom CSS in Streamlit

Resources (use in this order):

- **Read & code along** (1 hr): Streamlit: Build conversational apps → docs.streamlit.io/develop/tutorials/llms/build-conversational-apps
- **Read** (20 min): Anthropic Messages API reference → docs.claude.com/en/api/messages
- **Read** (15 min): Streamlit custom CSS → docs.streamlit.io/develop/concepts/configuration/theming
- **Reference** (for streaming code): Anthropic streaming examples → github.com/anthropics/anthropic-sdk-python#streaming

Step-by-step tasks:

1. New folder: `mkdir ai-chatbot && cd ai-chatbot`. Setup venv, `.env` (copy Anthropic key from ask-ai), `.gitignore`.
2. Install: `pip install streamlit anthropic python-dotenv`
3. Create `app.py`. Start from the Streamlit chatbot tutorial code (link above).
4. Adapt it to use Anthropic instead of OpenAI: `import anthropic`, use `client.messages.create()` instead of OpenAI's API.
5. Implement conversation memory: keep all messages in `st.session_state.messages`. On each user input, send the full messages list to Claude.
6. Add a system prompt selector in sidebar (`st.sidebar.selectbox`): 'Helpful Assistant', 'Code Reviewer', 'Study Tutor', 'Sarcastic Friend'. Each is a different system prompt string.
7. Add a model selector in sidebar: 'Claude Haiku 4.5' (cheap), 'Claude Sonnet 4.6' (balanced), 'Claude Opus 4.6' (smart but slow). Default to Sonnet.
8. Add streaming: use `client.messages.stream()` instead of `.create()`. Iterate and use `st.write_stream()` to render tokens as they arrive.
9. Add a 'Clear conversation' button in sidebar that resets `st.session_state.messages`
10. Custom CSS: add some style using `st.markdown` with `unsafe_allow_html=True`. Pick a color theme that's NOT the default Streamlit red. Add a custom title.
11. Test locally: `streamlit run app.py`. Have a 5-message conversation. Switch system prompts. Switch models. Clear and start over.
12. Create `requirements.txt`: `streamlit`, `anthropic`, `python-dotenv` (one per line)
13. Create GitHub repo 'ai-chatbot', push code (verify `.env` is NOT pushed)
14. Deploy to Streamlit Cloud: New app → select repo → file `app.py` → in 'Advanced settings' → Secrets → add `ANTHROPIC_API_KEY = "sk-ant-..."` (with quotes)
15. Wait for deploy. Test the live URL.
16. Update README with: live URL, screenshots, feature list, tech stack

Tools you'll use: Streamlit, Anthropic API, Streamlit Cloud

✓ Done when:

- ☐ Local chatbot remembers conversation across multiple messages

- ☐ Switching system prompt mid-conversation changes Claude's behavior
- ☐ Streaming works — you see tokens appear, not the full message at once
- ☐ Live deployed URL shows working chatbot to anyone who opens it
- ☐ README has live URL, 2 screenshots, and the GitHub link

DAY 10 · AI Content Studio (6 hours)

Focus: Build something Indian students will actually use — content for LinkedIn, cold emails, Insta captions.

What you'll learn today:

- Multi-template prompt design
- Using radio buttons / tabs for app modes
- Copy-to-clipboard and download buttons in Streamlit
- Designing for a specific user (in your head, picture YOUR friend using this)

Resources (use in this order):

- **Read** (15 min): Streamlit columns & layout → docs.streamlit.io/develop/api-reference/layout
- **Read** (15 min): Anthropic prompt templates → docs.claude.com/en/docs/build-with-claude/prompt-engineering/system-prompts
- **Look at** (15 min for inspiration): Copy.ai or Jasper homepage → copy.ai
- **Read** (10 min): st.download_button docs → docs.streamlit.io/develop/api-reference/widgets/st.download_button

Step-by-step tasks:

1. New folder: mkdir content-studio && cd content-studio. Same venv/.env/.gitignore setup.
2. Install: pip install streamlit anthropic python-dotenv
3. Design 4 content templates in your head: (1) LinkedIn post, (2) Cold email, (3) Blog post outline, (4) Instagram caption with hashtags
4. Create prompts.py file. For each template, write a system prompt that produces high-quality output for that format. Each prompt should be 100+ words and specify: tone, structure, length, what to avoid.
5. Create app.py. UI structure: title at top, then 4 tabs (one per template), then in each tab: input area on left, output area on right (use st.columns)
6. For each tab: text_area for user input (e.g., 'What's your post about?'), a tone selector (radio: Professional, Casual, Witty, Hinglish), word count selector (slider 50-500), Generate button
7. On Generate: combine the template's system prompt + the tone modifier + the word count + user input. Send to Claude. Display in output area.
8. Add a 'Copy to clipboard' button next to output (use st.code() which has a built-in copy button)
9. Add a 'Download as .txt' button using st.download_button
10. Add 'Hinglish' tone — useful for Indian users. Test it: ask for a LinkedIn post about productivity in Hinglish tone. Should output mix of Hindi and English.
11. Test all 4 templates end to end with different inputs
12. Style: add a clean header, app icon (emoji works), sidebar with 'About' info
13. requirements.txt: streamlit, anthropic, python-dotenv
14. Push to GitHub repo 'content-studio'
15. Deploy to Streamlit Cloud. Add ANTHROPIC_API_KEY in Secrets.
16. Update README with live URL + screenshots of each tab

Tools you'll use: Streamlit, Anthropic API

✓ Done when:

- ☐ All 4 tabs (LinkedIn, Email, Blog, Insta) produce coherent output
- ☐ Tone selector visibly changes output (Professional vs Hinglish are clearly different)
- ☐ Copy and Download buttons both work
- ☐ App is deployed and live on Streamlit Cloud
- ☐ Tested Hinglish output and confirmed it actually mixes languages naturally

DAY 11 · Image AI — Smart Document Reader (6 hours)

Focus: Multimodal AI (image + text) is hot in 2026. This shows recruiters you can do more than just text.

What you'll learn today:

- How to send images to LLMs (base64 encoding)
- Gemini Vision API (free)
- Anthropic Vision (for comparison)
- Building app modes for different use cases

Resources (use in this order):

- **Read** (20 min): Gemini Vision quickstart → ai.google.dev/gemini-api/docs/vision
- **Read** (20 min): Anthropic Vision → docs.claude.com/en/docs/build-with-claude/vision
- **Read** (15 min): Streamlit image upload → docs.streamlit.io/develop/api-reference/widgets/st.file_uploader
- **Try** (15 min): Gemini Vision in AI Studio (no code, just upload images) → aistudio.google.com

Step-by-step tasks:

1. New folder: `mkdir document-reader && cd document-reader`. Same setup.
2. Install: `pip install streamlit google-generativeai pillow python-dotenv`
3. Design 4 use case templates: (1) Handwritten notes → typed text, (2) Business card → structured contact info JSON, (3) Receipt → expense breakdown, (4) Whiteboard photo → structured notes
4. Create `app.py`. UI: title, mode selector (radio), file uploader for image, Process button, output area
5. When file is uploaded: display the image preview using `st.image()`
6. On Process: convert uploaded file to PIL Image, send to Gemini Vision API with a system prompt based on selected mode
7. For each mode, write a specific prompt. Example for receipts: 'Extract every line item from this receipt. Output as a clean markdown table with columns: Item, Quantity, Price. Total at the bottom.'
8. Display output as formatted markdown (`st.markdown`)
9. Add a 'Download as Markdown' button for the output
10. Test with real images: take a photo of your handwritten notes, a business card (real or printed), a recent receipt, and a whiteboard. Run each through.
11. Add error handling: no image uploaded, image too large, API errors
12. Push to GitHub repo 'document-reader', deploy to Streamlit Cloud (add `GEMINI_API_KEY` in Secrets)
13. README with live URL + 4 screenshots (one per mode)

Tools you'll use: Streamlit, Gemini Vision API, Pillow

✓ Done when:

- ☐ All 4 modes (notes, business card, receipt, whiteboard) work on real images
- ☐ Live URL deployed on Streamlit Cloud

- ☐ README has 4 before/after screenshots (image input → extracted output)
- ☐ You tested with at least 3 real images of your own

DAY 12 · AI Resume Reviewer (Portfolio Killer Project) (7 hours)

Focus: *This is the single most engagement-generating project in the program. Do this one really well.*

What you'll learn today:

- PDF text extraction with pypdf
- Structured output from LLMs (JSON mode / structured prompts)
- Scoring / rating logic with AI
- Showing 'AI thinking' to users with progress states

Resources (use in this order):

- **Read** (20 min): pypdf documentation → pypdf.readthedocs.io
- **Read** (20 min): Anthropic: Increase consistency (JSON output) → docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails/increase-consistency
- **Browse** (15 min): Real ATS scoring articles (search 'ATS resume scoring') → google.com/search?q=how+ATS+scores+resumes
- **Look at** (10 min): Existing resume reviewer apps for UI inspiration → google.com/search?q=ai+resume+reviewer+app

Step-by-step tasks:

1. New folder: `mkdir resume-reviewer && cd resume-reviewer`. Same setup.
2. Install: `pip install streamlit anthropic pypdf python-dotenv`
3. Create `utils.py` with a function `extract_text_from_pdf(file)` using pypdf
4. Create `app.py`. UI: title 'AI Resume Reviewer', two columns layout. Left column: PDF upload + Job Description text area. Right column: Analysis output.
5. Build prompt that asks Claude to return JSON with: (a) `overall_score` (0-100), (b) `ats_keyword_match` (0-100), (c) `missing_keywords` (list of strings), (d) `strengths` (list of 3-5 strings), (e) `improvements` (list of 5-7 specific suggestions), (f) `rewritten_summary` (improved summary section)
6. Important: use XML tags in prompt for resume and JD. Use prefill technique: end prompt with `{` so Claude continues as JSON. Parse with `json.loads()`.
7. On Analyze button: show 'Analyzing...' spinner. Extract PDF text. Send to Claude. Parse JSON. Display results.
8. Display the overall score as a big number with color coding: red <50, orange 50-75, green >75
9. Show ATS keyword match as a progress bar (`st.progress`)
10. Show missing keywords as red badges, strengths as green bullets, improvements as numbered list
11. Show rewritten summary in a code block users can copy
12. Add 'Export full analysis as markdown' download button
13. Polish UI: nice header, helpful empty state, error states (no PDF, no JD, parse failed)
14. Test with your own resume + a real job description from LinkedIn (any AI role)
15. Push to GitHub repo 'ai-resume-reviewer'. Deploy to Streamlit Cloud AND Hugging Face Spaces (we'll do HF properly tomorrow but try it today too)
16. README is critical for this one — make it the best you've written. Use Appendix A template. Add: live demo link, animated GIF of usage, technical architecture diagram (use draw.io)

Tools you'll use: Streamlit, Anthropic API, pypdf

✓ **Done when:**

- ☐ App accepts PDF resume and JD text, returns structured analysis
- ☐ Overall score shows with appropriate color (red/orange/green)
- ☐ Missing keywords, strengths, improvements all display clearly
- ☐ Live URL works for anyone (test by sharing with a friend)
- ☐ README is significantly better than your previous projects' READMEs (GIF, architecture, clear explanation)

DAY 13 · Production Polish & Hugging Face Spaces (5 hours)

Focus: Real engineering. Secrets, errors, READMEs — the things that separate a hobby project from a portfolio piece.

What you'll learn today:

- Environment variables vs secrets vs hardcoded values
- How Streamlit secrets work in deployment
- Deploying to Hugging Face Spaces (different from Streamlit Cloud)
- Writing a README that gets recruiters interested

Resources (use in this order):

- **Read** (20 min): Streamlit secrets management → docs.streamlit.io/develop/concepts/connections/secrets-management
- **Read** (30 min): Hugging Face Spaces overview → huggingface.co/docs/hub/spaces-overview
- **Read** (20 min): Deploy Streamlit to HF Spaces → huggingface.co/docs/hub/spaces-sdks-streamlit
- **Browse** (15 min): Awesome READMEs for inspiration → github.com/matiassingers/awesome-readme

Step-by-step tasks:

1. For each Week 2 project (chatbot, content-studio, document-reader, resume-reviewer), do this checklist:
2. Check 1: open .env file and verify NO hardcoded keys in app.py. All keys via os.getenv() or st.secrets
3. Check 2: try empty inputs (no text, no file). App should show friendly message, not crash.
4. Check 3: try API error (temporarily put wrong API key). App should catch error and show friendly message.
5. Check 4: try huge inputs (paste a 50,000 character text). App should either truncate gracefully or warn the user.
6. Now deploy each project to Hugging Face Spaces:
7. Go to huggingface.co/new-space. Name = your repo name. SDK = Streamlit. License = MIT. Hardware = CPU basic (free). Create.
8. HF gives you a git URL. In your project folder: `git remote add hf https://huggingface.co/spaces/YOUR_USERNAME/PROJECT_NAME` && `git push hf main`
9. On the HF Space page → Settings → Variables and secrets. Add your API keys as secrets (NOT variables).
10. Wait for build (~3-5 minutes). Get your HF URL (looks like huggingface.co/spaces/yourname/project)
11. Now rewrite ALL 4 README files using Appendix A template. Required sections: Title with emoji, one-line description, live demo links (both Streamlit AND HF), animated GIF or video demo, feature list, tech stack with badges, how to run locally, future improvements, your contact info
12. For GIFs: use Loom to record, then convert to GIF using ezgif.com (free) and embed in README
13. Commit and push to all 3 places (GitHub, Streamlit Cloud, Hugging Face)

Tools you'll use: Hugging Face Spaces, Streamlit Cloud, GitHub, Loom, ezgif

✓ **Done when:**

- ☐ All 4 Week 2 projects pass the 4 production checks (no hardcoded keys, handles empty input, handles API errors, handles huge inputs)
- ☐ All 4 projects are deployed on BOTH Streamlit Cloud AND Hugging Face Spaces (so 8 live URLs total)
- ☐ All 4 READMEs are polished with: live links, GIF demo, architecture, tech stack
- ☐ If you click any GitHub repo, the README looks better than 90% of student repos you can find on GitHub

DAY 14 · Week 2 LinkedIn Showcase (5 hours)

Focus: *Pick your best work and present it loudly. This is the post that gets DMs from recruiters.*

What you'll learn today:

- How to structure a 'showcase' LinkedIn carousel
- Hook writing (the first slide / first line)
- Adding 2 polish features to one project (the 'one more thing' move)

Resources (use in this order):

- **Look at** (30 min): Top LinkedIn AI builder posts (search and study what gets traction) → [linkedin.com/search/results/content/?keywords=%23AIBuilder](https://www.linkedin.com/search/results/content/?keywords=%23AIBuilder)
- **Use** (as needed): Canva carousel templates → [canva.com/templates/?query=linkedin+carousel](https://www.canva.com/templates/?query=linkedin+carousel)
- **Reference** (for caption template): Appendix B in this doc → [see Appendix B below](#)

Step-by-step tasks:

1. Pick your favorite project from Week 2. Likely the Resume Reviewer (it's the most universal).
2. Add 2 new features to it. Examples: (1) generate a tailored cover letter, (2) suggest 3 better job titles to apply for, (3) export the rewritten resume as a downloadable .txt
3. Make sure new features work end-to-end. Update the live deploy.
4. Now the carousel. Open Canva → search 'LinkedIn carousel' → pick a clean modern template.
5. 10 slides: (1) Hook: 'I built 4 AI apps in 7 days as a 3rd year student. Here's everything.' (2) The problem you wanted to solve (one sentence per project) (3-6) One slide per app: name, screenshot, key feature, live link (7) The tech stack visual (8) The hardest thing you learned (9) What's next week (10) Final CTA: 'Drop a comment if you want me to share the code'
6. Use Appendix B (Week 2 template) for the caption
7. Post on LinkedIn. Tag #BuildInPublic #AIEngineer #100DaysOfCode #StudentBuilder
8. First comment: drop links to all 4 projects (Streamlit URLs)
9. Engage: spend 30 minutes replying to every comment. Ask questions back to commenters.

Tools you'll use: Canva, LinkedIn

✓ **Done when:**

- ☐ Your chosen project has 2 new features that work in production
- ☐ LinkedIn carousel of 10 slides is published
- ☐ All 4 project URLs are in the first comment
- ☐ You've replied to every comment received within 4 hours of posting

Week 3: RAG & Vector Databases

Goal: Learn the most in-demand AI engineering skill of 2026 — Retrieval-Augmented Generation (RAG). This is what makes AI apps actually useful with real data. By end of Week 3 you'll have a 'Chat with your PDFs' app and a 'Chat with YouTube videos' app deployed.

DAY 15 · Embeddings & Vector DBs — Theory (5 hours)

Focus: *Understand the magic behind 'AI that knows your data'.*

What you'll learn today:

- What embeddings are (intuition, not math)
- Cosine similarity (the only formula you really need)
- What a vector database does and why it exists
- The full RAG pipeline at a high level

Resources (use in this order):

- **Read** (1 hr): Vicki Boykis: What are Embeddings? (free PDF — this is THE best intro) → vickiboykis.com/what_are_embeddings/
- **Read** (30 min): Pinecone Learn: Vector Database 101 → pinecone.io/learn/vector-database/
- **Watch** (20 min): RAG explained by IBM Technology → youtube.com/watch?v=T-D1OfcDW1M
- **Try** (15 min): Embedding Projector — visualize embeddings → projector.tensorflow.org
- **Reference** (skim): Anthropic: Embeddings overview → docs.claude.com/en/docs/build-with-claude/embeddings

Step-by-step tasks:

1. Read the Vicki Boykis PDF (chapters 1-3 minimum). Take notes on: what a vector is, what 'similar' means in vector space, why we need embeddings at all.
2. Watch the IBM RAG video. Pause and draw the RAG pipeline on paper: User Question → Embed → Search Vector DB → Get Relevant Chunks → Pass to LLM → Answer.
3. Open projector.tensorflow.org. Click the 'Word2Vec All' dataset. Search for 'king'. Look at the nearest neighbors. Search for 'India'. Look at neighbors. Get an intuition for what 'close in vector space' means.
4. Create a new file in your ai-30-day-journey repo: rag-notes.md
5. Write 300 words covering: (a) what an embedding is, (b) why we use cosine similarity, (c) what a vector database stores, (d) the 5 steps of RAG, (e) one example where RAG is better than just prompting
6. Draw the RAG pipeline. Use draw.io (free at draw.io or app.diagrams.net). Export as PNG. Add to your repo as rag-pipeline.png.
7. Embed the diagram in rag-notes.md using markdown: ![RAG Pipeline](rag-pipeline.png)
8. Commit and push: `git add . && git commit -m "Day 15: RAG and embeddings notes" && git push`

Tools you'll use: Vicki Boykis PDF, TensorFlow Projector, draw.io, VS Code

✓ Done when:

- ☐ rag-notes.md exists in your repo with the 5 sections written in your own words
- ☐ rag-pipeline.png is a clean diagram you made yourself (not copied)
- ☐ You can draw the RAG pipeline on a whiteboard from memory if asked in an interview
- ☐ You can explain cosine similarity in one sentence without looking

DAY 16 · ChromaDB Hands-On (5 hours)

Focus: *Stop reading. Start storing actual vectors and searching them.*

What you'll learn today:

- ChromaDB basics (the simplest vector DB to learn on)
- Creating collections, adding documents, querying
- How embedding functions work in Chroma
- Persistent vs in-memory Chroma

Resources (use in this order):

- **Read** (30 min): Chroma Getting Started → docs.trychroma.com/getting-started
- **Read & code along** (1 hr): Chroma Usage Guide → docs.trychroma.com/usage-guide
- **Reference** (as needed): Chroma cookbook → cookbook.chromadb.dev

Step-by-step tasks:

1. New folder: `mkdir vector-search && cd vector-search`. Setup `venv`.
2. Install: `pip install chromadb sentence-transformers`
3. Create `play.py` to experiment. Start with the example in Chroma docs: create a client, create a collection, add 5 documents, query with a text and see results.
4. Now build something real: create `search_engine.py`. It will be a semantic search over Bollywood movies.
5. Get 100 movie plots: easiest way is to ask Claude.ai 'Give me 100 Bollywood movie titles and their 2-sentence plots, as a Python list of dicts with keys title and plot.' Paste into your code as a Python list.
6. Initialize a persistent Chroma client: `chromadb.PersistentClient(path='./chroma_db')`
7. Create a collection called 'movies'. Add all 100 movies to it. Each document = plot. Each metadata = {'title': title}. ID = index.
8. Write a query function: `query_movies(question, n=5)` that returns top 5 most relevant titles + plots
9. Test queries: 'romantic college story', 'underdog sports', 'family drama with twist ending', 'thriller about politics'
10. Print results nicely with title, plot, and similarity score
11. Build a Streamlit UI on top: `app.py` with a search box, `n=5` default, results displayed as cards
12. Test: type any natural language query, get semantically similar movies. This is RAG without the LLM step!
13. Push to GitHub repo 'movie-semantic-search'. Deploy to Streamlit Cloud.

Tools you'll use: ChromaDB, Streamlit, sentence-transformers

✓ Done when:

- ☐ Local `search_engine.py` works: queries return semantically similar movies
- ☐ Streamlit app deployed and live with 100 indexed movies
- ☐ You understand the difference between metadata search (exact match on title) and semantic search (find similar plots)
- ☐ You can explain what `chroma.add()` and `chroma.query()` each do

DAY 17 · Chat with PDF — Part 1 (Data Pipeline) (6 hours)

Focus: *Build the indexing half of a real RAG app. The chat half comes tomorrow.*

What you'll learn today:

- Text chunking strategies (and why naïve chunking fails)
- Why chunk size and overlap matter
- How to handle multi-page PDFs

- Metadata that you'll need later (page numbers, chunk index)

Resources (use in this order):

- **Read** (30 min): Pinecone: Chunking strategies → pinecone.io/learn/chunking-strategies/
- **Read** (20 min): Recursive text splitter logic explained → python.langchain.com/docs/modules/data_connection/document_transformers/recursive_text_splitter
- **Read** (15 min): pypdf docs (reading PDFs) → pypdf.readthedocs.io
- **Reference** (as needed): ChromaDB add() reference → docs.trychroma.com/reference/Collection

Step-by-step tasks:

1. New folder: mkdir chat-with-pdf && cd chat-with-pdf. Same setup (venv, .env, .gitignore).
2. Install: pip install streamlit chromadb pypdf anthropic python-dotenv
3. Create indexer.py. This is the offline data pipeline.
4. Step 1 — read_pdf(file_path): use pypdf to extract text from each page. Return list of (page_num, text) tuples.
5. Step 2 — chunk_text(text, chunk_size=500, overlap=50): split text into ~500 character chunks with 50 character overlap. Don't break mid-word. Simple approach: split on sentences, then group sentences until chunk_size is hit.
6. Step 3 — embed_and_store(chunks, collection): pass chunks to a Chroma collection. Metadata for each chunk = {'page': page_num, 'chunk_index': i}.
7. Wire it together in indexer.py main: takes a PDF path and a collection name as CLI args. Reads PDF → chunks → stores in Chroma.
8. Test: download any public PDF (e.g., a research paper from arxiv.org). Run: python indexer.py paper.pdf my_paper
9. Verify in Python REPL: client = chromadb.PersistentClient(...); c = client.get_collection('my_paper'); c.count(). Should be > 0.
10. Test queries directly: c.query(query_texts=['what is the main finding'], n_results=3). Should return relevant chunks.
11. Now wrap in a small Streamlit UI: upload_app.py that lets user upload PDF + name the collection, calls your indexer functions, shows progress
12. Test with 2 different PDFs: one academic paper, one textbook chapter, see results differ
13. Commit progress: git init, push to new GitHub repo 'chat-with-pdf'

Tools you'll use: pypdf, ChromaDB, Streamlit

✓ Done when:

- ☐ Running indexer.py on a PDF creates a Chroma collection with chunks
- ☐ You can query the collection in Python and get relevant chunks back
- ☐ Streamlit upload UI works locally
- ☐ You can articulate why we use overlap between chunks (hint: avoids losing context at boundaries)

DAY 18 · Chat with PDF — Part 2 (RAG Query Loop) (6 hours)

Focus: The other half. User asks question → fetch chunks → ask Claude with context → answer with citations.

What you'll learn today:

- The retrieval → augmentation → generation flow
- Including source citations in answers
- Conversation memory ON TOP of RAG (a common interview question)

- When RAG fails (and what to do about it)

Resources (use in this order):

- **Re-read** (15 min): Anthropic: contextual retrieval (good prompting patterns) → anthropic.com/news/contextual-retrieval
- **Read** (20 min): Anthropic system prompts → docs.claude.com/en/docs/build-with-claude/prompt-engineering/system-prompts
- **Reference** (as needed): Streamlit chat tutorial (you used this in Day 9) → docs.streamlit.io/develop/tutorials/llms/build-conversational-apps

Step-by-step tasks:

1. Open chat-with-pdf folder from yesterday. Create chat_app.py as the main app.
2. UI: Sidebar with: PDF uploader, 'Index PDF' button, collection selector dropdown. Main area: chat interface (like Day 9 chatbot).
3. On 'Index PDF' click: call your indexer code from Day 17. Show progress. Reload the collection selector with the new collection name.
4. Implement the RAG query function: query_rag(question, collection): (1) embed question, (2) chroma.query() for top 5 chunks, (3) format chunks with their page numbers, (4) build prompt: system='Answer based ONLY on the context. Cite the page number after each fact. If not in context, say so.' user='<context>{chunks}</context><question>{question}</question>', (5) send to Claude, (6) return response + the chunks used
5. Wire it into the chat loop: every user message goes through query_rag. Display the answer. Below the answer, add an expander 'View source chunks' that shows which chunks were used.
6. Add conversation memory: keep messages in session_state. BUT for the actual RAG query, only use the LATEST user question to retrieve. Then send the conversation history + retrieved chunks to Claude. (This is a key pattern — interview question!)
7. Add a 'Clear chat' button. Add a 'Switch PDF' option that lets user switch between indexed collections without re-indexing.
8. Edge cases: empty question, no PDF indexed yet, irrelevant question (test by asking something not in the PDF — should respond 'I don't see that in the document')
9. Polish: nice header, app icon, loading spinners, error messages
10. Test with a real document: download your college's policies PDF (or any 20+ page PDF). Index it. Ask 5 questions. Verify citations.
11. requirements.txt: streamlit, chromadb, pypdf, anthropic, python-dotenv
12. Push to GitHub. Deploy to Streamlit Cloud (add ANTHROPIC_API_KEY in Secrets). Note: Chroma's persistent files won't transfer, so deployed app starts fresh — that's OK for now.
13. Update README using Appendix A. Include: live URL, architecture diagram (extend Day 15's RAG diagram), 'how it works' explainer, GIF demo

Tools you'll use: Streamlit, ChromaDB, Anthropic API, pypdf

✓ Done when:

- ☐ App lets user upload PDF, index it, then chat with it
- ☐ Answers include page-number citations like 'On page 4...'
- ☐ Asking something NOT in the PDF triggers honest 'I don't see this in the document' response
- ☐ Source chunks are viewable in an expander below each answer
- ☐ Live URL deployed and demoable
- ☐ README has architecture diagram + GIF demo

DAY 19 · Chat with YouTube (5 hours)

Focus: Same RAG pattern, different data source. Shows you can adapt the architecture.

What you'll learn today:

- Reusing your RAG code with a new ingestion source
- Handling longer transcripts (chunking matters more here)
- Adding timestamp citations (YouTube-specific touch)

Resources (use in this order):

- **Read** (10 min): youtube-transcript-api docs (you used this on Day 6) → pypi.org/project/youtube-transcript-api/
- **Reference** (as needed): Your own chat-with-pdf code from Day 18 → *your local folder*

Step-by-step tasks:

1. New folder: `mkdir chat-with-youtube && cd chat-with-youtube`. Same setup.
2. Install: `pip install streamlit chromadb youtube-transcript-api anthropic python-dotenv`
3. Copy your indexer pattern from chat-with-pdf. Modify: instead of reading PDF pages, fetch YouTube transcript using `youtube_transcript_api`.
4. `youtube_transcript_api` returns list of dicts: `{text, start, duration}`. Combine the text fields, but KEEP the timestamps as metadata.
5. Chunk strategy: group consecutive transcript entries until you hit ~500 characters per chunk. Store the start timestamp of the chunk's first entry as metadata.
6. Indexer takes: YouTube URL → extract video ID → fetch transcript → chunk with timestamps → store in Chroma. Collection name = video ID.
7. Build the Streamlit app: paste a YouTube URL → click Index → chat with it. Same chat pattern as Day 18.
8. Citation enhancement: in the prompt, ask Claude to cite timestamps like [5:23]. Then in your UI, parse those and turn them into clickable YouTube links that jump to that timestamp (URL format: `youtube.com/watch?v=VIDEO_ID&t=323s`)
9. Test with a 15-minute video (a tech talk or tutorial). Ask 5 questions. Verify timestamp links work.
10. Test edge case: video with NO transcript available — show a friendly error.
11. Push to GitHub. Deploy to Streamlit Cloud.
12. Update README with the live URL and a GIF showing the timestamp citations working

Tools you'll use: youtube-transcript-api, ChromaDB, Streamlit, Anthropic API

✓ Done when:

- ☐ Paste a YouTube URL → app indexes the transcript → you can chat with it
- ☐ Answers include timestamp citations like [5:23]
- ☐ Timestamps are clickable and open the video at that time
- ☐ Handles videos without transcripts gracefully
- ☐ Live URL deployed on Streamlit Cloud

DAY 20 · LangChain Comparison — When to Use Frameworks (5 hours)

Focus: *Rebuild a small RAG with LangChain. Form an opinion. (Opinion required: 'frameworks help' or 'frameworks hurt' — interview will ask.)*

What you'll learn today:

- LangChain core concepts (LCEL, retrievers, chains)
- When LangChain saves time vs adds complexity
- The framework vs raw API debate (real industry discussion)

Resources (use in this order):

- **Read** (1 hr): LangChain Quickstart → python.langchain.com/docs/get_started/introduction

- **Read** (30 min): LangChain RAG tutorial → python.langchain.com/docs/use_cases/question_answering/quickstart
- **Read** (20 min for the other side): 'Why I quit LangChain' (search Hacker News for varied opinions) → news.ycombinator.com/from?site=octomind.dev
- **Reference** (skim): LangChain integrations with Anthropic → python.langchain.com/docs/integrations/chat/anthropic

Step-by-step tasks:

1. New folder: mkdir langchain-experiment && cd langchain-experiment. Same setup.
2. Install: pip install langchain langchain-anthropic langchain-chroma langchain-community pypdf python-dotenv
3. Build a minimal RAG with LangChain following their RAG quickstart. Use the same PDF you used in Day 18.
4. Goal: produce the same Q&A behavior as your Day 18 app, but using LangChain primitives.
5. Compare line counts: your raw Day 18 implementation vs LangChain version. Note differences.
6. Time how long it takes to add a feature in each: 'Add a setting for top-k retrieval'. Add it to your raw version (5 min?). Add to LangChain version (depends on how well you know LangChain).
7. Try one more LangChain-specific thing: use LangChain's RecursiveCharacterTextSplitter to chunk. Compare to your custom chunking.
8. Create comparison.md in your repo. Write 400 words: (a) what was easier with LangChain, (b) what was harder/more confusing, (c) when YOU would use LangChain vs raw API, (d) your overall opinion
9. Push everything to a new GitHub repo: 'rag-framework-comparison'
10. README should be a fair comparison post — this is portfolio-grade content. Title: 'I built the same RAG twice — once raw, once with LangChain. Here's what I learned.'

Tools you'll use: LangChain, ChromaDB, Anthropic API

✓ Done when:

- ☐ Working LangChain-based RAG runs locally and answers questions on a PDF
- ☐ comparison.md exists with a thoughtful, opinionated comparison
- ☐ You have a clear answer to 'when would you use LangChain' if asked in interview
- ☐ GitHub README presents this as a thoughtful comparison, not a how-to

DAY 21 · Week 3 Showcase + Mid-Program Reflection (5 hours)

Focus: Show the world your RAG work. Reflect on what's been hard. Set up Week 4 for success.

What you'll learn today:

- How to talk about RAG in a way non-technical people understand
- Writing a reflection post (more engaging than 'here's what I built' posts)

Resources (use in this order):

- **Reference** (as needed): Appendix B (Week 3 LinkedIn template) → [see Appendix B](#)
- **Reference** (as needed): Your previous LinkedIn carousels for style consistency → [your own LinkedIn](#)

Step-by-step tasks:

1. Pick the best of your 3 Week 3 apps to feature. Likely chat-with-pdf or chat-with-youtube.
2. Record a 90-second Loom demo showing: paste URL/upload PDF → index → ask a question → get answer with citation → click citation → it works
3. Upload Loom video. Get shareable link. Also export as MP4 for LinkedIn (LinkedIn loves native video).

4. Write a Hashnode blog post: 'I built Chat-with-PDF this week. Here's what RAG actually is (in plain English).'
5. Structure: (1) The problem (LLMs don't know your data) (2) The 'aha' moment (embeddings = similarity in vector space) (3) The 5 steps of RAG with a diagram (4) What I built (5) The mistake I made and how I fixed it (6) Code snippets of the key parts (7) Live demo link
6. Embed the diagram from Day 15 in the blog post.
7. Publish blog post.
8. Create Week 3 LinkedIn carousel (10 slides) following Appendix B template
9. Slide ideas: (1) Hook 'RAG is the AI skill nobody teaches you. I built 3 apps with it this week.' (2-4) Each Week 3 project (5) The RAG pipeline diagram from Day 15 (6) Surprising thing I learned (7) Hardest debugging story (8) Stats so far (X projects, Y commits, Z LinkedIn followers gained) (9) What's coming Week 4 (10) CTA
10. Post LinkedIn carousel. In FIRST COMMENT add the Loom demo video link + Hashnode blog link.
11. Reflection time. Open a personal note (not for posting). Answer honestly: (a) Am I on track? (b) Which week was hardest and why? (c) What do I want to focus on in Week 4? (d) Did I skip anything I should go back to?
12. If you skipped anything important: schedule 2 hours on a Saturday/Sunday this week to fill gaps

Tools you'll use: Loom, Hashnode, Canva, LinkedIn

✓ **Done when:**

- ☐ 90-second Loom demo recorded and uploaded
- ☐ Hashnode blog post 'What RAG actually is' published
- ☐ LinkedIn carousel posted with 10 slides
- ☐ Demo video + blog link in first comment
- ☐ Personal reflection note written (not for posting, just honest self-check)

Week 4: Agents, Capstone & Career

Goal: Build an AI agent (the hottest topic in 2026), ship one ambitious capstone project, and polish your career assets. By Day 30 you'll be ready to apply to AI internships and roles with confidence.

DAY 22 · Understanding AI Agents (5 hours)

Focus: *Agents = LLMs that can take actions. The defining concept of 2026 AI engineering.*

What you'll learn today:

- What 'tool use' or 'function calling' means
- The agent loop (think → act → observe → repeat)
- Difference between an agent and a workflow
- When agents help vs when they're overkill

Resources (use in this order):

- **Read** (1 hr (MUST READ)): Anthropic: Building effective agents → anthropic.com/research/building-effective-agents
- **Read** (30 min): Anthropic tool use overview → docs.claude.com/en/docs/agents-and-tools/tool-use/overview
- **Read** (20 min): Tool use examples → docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use
- **Watch** (15 min): Anthropic tool use video walkthrough (find recent one on YouTube) → youtube.com/results?search_query=anthropic+tool+use+claude

Step-by-step tasks:

1. Read the 'Building effective agents' essay slowly. It's the most important read in this program. Take notes on: the difference between workflows and agents, the 5 patterns they describe, the augmented LLM.
2. New folder: `mkdir agents-101 && cd agents-101`. Setup venv.
3. Install: `pip install anthropic python-dotenv`
4. Create `simple_tools.py`. Define 3 dummy tools as Python functions: `get_weather(city)`, `calculate(expression)`, `get_current_time()`. Make them work (use `requests`, `eval`, `datetime` — keep it simple).
5. Now make Claude use these tools. Following the tool use docs, define tools in the Anthropic API format (name, description, input_schema as JSON Schema).
6. Build a basic agent loop: (1) Send user question + tools list, (2) If Claude returns `tool_use` blocks, execute the tools, (3) Send results back as `tool_result`, (4) Repeat until Claude returns text (no more tool calls), (5) Print final answer.
7. Test with: 'What's the weather in Bangalore right now?', 'What's $(245 * 13) + \sqrt{81}$?', 'What's the time difference between Bangalore and New York?' — Claude should pick the right tool each time.
8. Print the trace: each tool call, the input, the output, then the final answer. This is your 'agent transcript'.
9. Add a `max_iterations` safety limit (don't let the loop run more than 10 times)
10. Push to GitHub repo 'agents-101'. README explains: what an agent is, the loop, and shows a transcript of a real run.

Tools you'll use: Anthropic API, Python requests, Python datetime

✓ Done when:

- ☐ You can articulate the agent loop in plain English: think, act, observe, repeat
- ☐ Your simple agent uses all 3 tools correctly when asked the right questions
- ☐ GitHub README has a real transcript example showing the agent reasoning

- ☐ You can explain when to use an agent vs a workflow (Anthropic's distinction)

DAY 23 · Research Assistant Agent (7 hours)

Focus: *First real agent. Give it a research question, it browses the web, returns a structured report.*

What you'll learn today:

- Multi-tool agent design (web search + web fetch + summarize)
- Designing tool descriptions that the LLM actually understands
- Handling agent failures (loops, hallucinated tool calls)
- Streaming agent traces to UI

Resources (use in this order):

- **Re-read** (30 min): Anthropic tool use guide (deeper this time) → docs.claude.com/en/docs/agents-and-tools/tool-use/overview
- **Read** (20 min): DuckDuckGo Search Python package → pypi.org/project/duckduckgo-search/
- **Read** (15 min): BeautifulSoup quickstart → crummy.com/software/BeautifulSoup/bs4/doc/
- **Reference** (as needed): Anthropic prompt caching (saves cost for agents) → docs.claude.com/en/docs/build-with-claude/prompt-caching

Step-by-step tasks:

1. New folder: `mkdir research-agent && cd research-agent`. Same setup.
2. Install: `pip install streamlit anthropic duckduckgo-search beautifulsoup4 requests python-dotenv`
3. Build 3 tools: `search_web(query)` → returns top 5 search results (title, url, snippet) using `duckduckgo-search`. `fetch_url(url)` → fetches page, extracts main text with `BeautifulSoup`, returns first 5000 chars. `write_section(heading, content)` → just stores in a list (lets agent build up final report).
4. Define these 3 tools in Anthropic format with good descriptions. The descriptions are critical — Claude reads them to decide when to use each.
5. Build the agent loop. System prompt: 'You are a research assistant. Given a topic, search the web, fetch the most relevant 3-5 pages, and write a structured report with 4 sections: Overview, Key Findings, Caveats, Sources. Use `write_section` to build the report progressively.'
6. Loop: while Claude returns `tool_use` → execute → send `tool_result` back → continue. When Claude returns text without tools, that's the final answer.
7. Add a safety limit: max 15 iterations or it stops.
8. Build Streamlit UI: text input for topic, 'Start Research' button, then below it a live trace area that shows each step: 'Searching for X...', 'Fetched page Y...', 'Writing section Z...'
9. Show the final report as nicely formatted markdown at the bottom.
10. Add 'Download as Markdown' button for the final report.
11. Polish: a nice header, what-can-it-do examples, status indicator while running.
12. Test with 3 topics: 'Latest developments in fusion energy 2026', 'Comparison of vector databases for production use', 'Career paths for Indian engineering students in AI'
13. Push to GitHub repo 'research-agent'. Deploy to Streamlit Cloud (add `ANTHROPIC_API_KEY` in secrets).
14. README is critical for this one — agents are HOT in 2026, this is the project that will get DMs. Use Appendix A template. Add: live URL, animated GIF of the agent thinking + searching, architecture diagram, list of tools.

Tools you'll use: Streamlit, Anthropic API, DuckDuckGo Search, BeautifulSoup

✓ Done when:

- ☐ Given any research topic, the app produces a structured report in 30-90 seconds

- ☐ User sees live trace: search, fetch, write — they can watch the agent 'think'
- ☐ Final report has 4 sections (Overview, Key Findings, Caveats, Sources) with real URLs cited
- ☐ Live URL deployed and tested with 3 real topics
- ☐ README has GIF showing the agent in action

DAY 24 · Capstone Day 1 — Plan & Spec (5 hours)

Focus: *Planning. The next 4 days are your capstone. A bad plan today = a painful Day 25-27.*

What you'll learn today:

- How to write a project spec
- Scoping ambition correctly (build less than you think you can)
- MVP vs polished version trade-offs

Resources (use in this order):

- **Read** (as needed): Capstone Project Ideas (in this document) → [see Capstone section below](#)
- **Read** (10 min): Writing a good MVP spec (search for any quick guide) → [google.com/search?q=how+to+write+mvp+spec](https://www.google.com/search?q=how+to+write+mvp+spec)

Step-by-step tasks:

1. Read the 7 Capstone Project Ideas section in this document. Pick ONE that excites you most.
2. Don't pick what's easiest — pick what's most useful for YOU. Will you use it after the program? That's the right one.
3. Create a new GitHub repo for your capstone. Name it well — this goes on your resume. Examples: 'cracai-prep', 'lecture-genie', 'jobhunt-copilot'.
4. Create SPEC.md in the repo. Write a 1-page spec covering: (1) Problem you're solving (2 sentences), (2) Target user (1 sentence), (3) Core features for MVP (3-5 bullets max!), (4) Stretch features for if time allows (3 bullets), (5) Tech stack you'll use, (6) Why this matters (your motivation, 2 sentences)
5. Sketch the UI on paper or in Figma (free at figma.com). Just 3-4 screens, rough wireframes. Take a photo and add to repo as wireframes.png.
6. Plan the data flow on paper. What's the input, what processes it, what's the output? Draw the boxes and arrows. Add to repo as data-flow.png.
7. Now plan the next 3 days (Days 25-27): Day 25 = core backend logic + key APIs. Day 26 = UI + integration. Day 27 = deploy + polish + README + LinkedIn launch. Write this plan into the SPEC.md.
8. Critical: identify the SINGLE highest-risk technical question. Example: 'Can ChromaDB handle 1000 documents?' or 'Will this prompt format work reliably?'. Spend the last hour TODAY doing a quick prototype to answer that risk. Don't build the app yet — just answer the risky question.
9. Commit SPEC.md, wireframes.png, data-flow.png, and any prototype code. Push.

Tools you'll use: GitHub, Figma or paper sketches, draw.io

✓ Done when:

- ☐ SPEC.md exists with all 6 sections filled in
- ☐ UI wireframes and data flow diagrams are in the repo
- ☐ You've identified the riskiest technical unknown and answered it today
- ☐ You have a clear day-by-day plan for the next 3 days
- ☐ You're excited about what you're building (this matters — if not, change projects NOW)

DAY 25 · Capstone Day 2 — Core Build (8 hours)

Focus: *Today is build day. No meetings, no distractions. Just code.*

What you'll learn today:

- Maintaining focus on the core feature, not the polish
- Debugging discipline (commit often, never go more than 30 min without a working state)

Resources (use in this order):

- **Reference** (as needed): Your past project READMEs for code patterns → [your GitHub](#)
- **Reference** (as needed): Anthropic API docs → docs.claude.com/en/api
- **Use** (actively): Claude.ai or Cursor for coding help — describe what you want, paste code, ask for debugging help → claude.ai

Step-by-step tasks:

1. Open your capstone folder. Setup venv, install dependencies from your spec.
2. Build the CORE backend logic first. No UI yet. Just functions that do the AI work — API calls, data processing, RAG indexing, agent loop, whatever your project needs.
3. Test each function in isolation. Use a scratch file (test.py) to call each function and verify it works.
4. Commit at every working milestone. Aim for 10+ commits today. Commit messages like 'Add PDF chunking function' or 'Get tool use loop working'.
5. If you get stuck for 45 minutes: STOP. Ask Claude.ai. Paste your code, paste the error, ask 'why is this happening?'. 99% of the time, it solves it.
6. If Claude can't solve it in 30 minutes: search the exact error message on Google. Stack Overflow is your friend.
7. If still stuck after 90 minutes total: ping me on WhatsApp. Don't waste a day stuck.
8. By end of day: you should have a CLI or notebook that takes the core input and produces the core output. UI comes tomorrow.

Tools you'll use: Cursor or VS Code, Anthropic API, Claude.ai as tutor, Git

✓ **Done when:**

- ☐ Core backend functions work in isolation (tested in scratch script)
- ☐ At least 10 commits pushed to your capstone repo today
- ☐ You can demonstrate the end-to-end flow via a Python script (no UI yet)
- ☐ You haven't been stuck on any single problem for more than 90 min — you asked for help when needed

DAY 26 · Capstone Day 3 — UI & Integration (8 hours)

Focus: *Wrap the backend in Streamlit. Make it look good. Handle the errors.*

What you'll learn today:

- UI design for AI apps (loading states, error states, empty states)
- Making your app accessible to non-technical users

Resources (use in this order):

- **Reference** (as needed): All your Week 2 Streamlit apps for code patterns → [your GitHub](#)
- **Look at** (30 min for inspiration): Top Streamlit apps in the Gallery → streamlit.io/gallery

Step-by-step tasks:

1. Build app.py — the Streamlit UI that wraps your backend logic.
2. Start ugly: just put the widgets in roughly the right places and wire them to your functions. Confirm end-to-end works.
3. Then polish: header with emoji and title, sidebar with About/settings, clean layout using columns and expanders.

4. Add ALL these states: (1) Empty state — what user sees before they do anything (with an example or call to action), (2) Loading state — st.spinner while AI is working, (3) Error state — if API fails, show friendly message not stack trace, (4) Success state — clear visual feedback when something worked
5. Test on edge cases: empty input, huge input, malformed input, no internet (kill wifi briefly to test)
6. Get someone else to test it. Show your laptop to a friend or family member. Don't explain — just say 'try to use this'. Watch where they get confused. Fix those things.
7. If you have stretch features from your spec — add ONE of them if you have time. Save 1 hour for deploy + README tomorrow.
8. Commit. Push. End the day with a working, polished, locally-running app.

Tools you'll use: Streamlit, Cursor/VS Code, Friends/family for testing

✓ **Done when:**

- ☐ Streamlit app runs locally with no errors
- ☐ All 4 UI states implemented (empty, loading, error, success)
- ☐ At least one non-technical person has tried it without your help
- ☐ You've fixed the top 2-3 confusing things they hit

DAY 27 · Capstone Day 4 — Deploy, README, Launch (6 hours)

Focus: *Ship it. The world needs to see this.*

What you'll learn today:

- Production deploy checklist
- Writing a project launch post that gets attention

Resources (use in this order):

- **Reference** (as needed): Appendix A — README template → [see Appendix A](#)
- **Reference** (as needed): Appendix B — Launch post template → [see Appendix B](#)
- **Use** (as needed): ezgif.com to convert Loom MP4 to GIF → ezgif.com/video-to-gif

Step-by-step tasks:

1. Production checklist: (1) No hardcoded API keys (use st.secrets or os.getenv), (2) requirements.txt has every dependency, (3) .gitignore excludes .env, venv/, __pycache__/, (4) error handling for every external API call, (5) reasonable timeouts on web fetches
2. Deploy to Streamlit Cloud: New app → select repo → app.py → Advanced settings → Secrets → add your keys
3. ALSO deploy to Hugging Face Spaces (you learned this on Day 13). Two deployments = backup if one breaks.
4. Once both URLs work, write the README using Appendix A template. Include: title with emoji, live demo URL (both), one-line description, animated GIF, feature list with screenshots, tech stack with badges, architecture diagram, how to run locally, future improvements, your contact info (LinkedIn, GitHub, email)
5. Record a 2-minute Loom demo. Walk through the app from a user's perspective. End by saying 'Built by Jathin, 3rd year engineering student. Link in description.'
6. Convert Loom video to GIF using ezgif.com. Add GIF to top of README.
7. Take 4-5 screenshots of key moments. Add to README.
8. Write LinkedIn launch post using Appendix B (Capstone Launch template). This is your most important post in the program.
9. Post on LinkedIn at 8 AM IST (highest engagement). Tag #BuildInPublic #AIEngineer #Capstone #IndianTech #StudentBuilder

10. First comment: live demo URL + GitHub URL + Loom video URL
11. Post in 3 relevant places: (1) Your college's WhatsApp/Discord group, (2) r/india_startups or relevant subreddit, (3) Hacker News if it's interesting enough (Show HN)
12. Spend the evening replying to every comment, DM, and reaction. Engagement window is the first 4 hours.

Tools you'll use: Streamlit Cloud, Hugging Face Spaces, Loom, ezgif, LinkedIn

✓ **Done when:**

- ☐ Capstone deployed at 2 URLs (Streamlit Cloud + HF Spaces) and both work
- ☐ README is the best you've written — GIF, screenshots, architecture, contact info
- ☐ 2-minute Loom demo recorded and linked from README
- ☐ LinkedIn launch post published with all 3 links in first comment
- ☐ Posted in at least 1 community outside LinkedIn
- ☐ Replied to every comment in the first 4 hours after posting

DAY 28 · Resume & GitHub Profile Polish (5 hours)

Focus: Turn your work into a recruiter magnet. This is where the 30 days converts to interviews.

What you'll learn today:

- What recruiters skim resumes for in the first 6 seconds
- STAR format for project bullets
- GitHub README profile (the meta-profile)

Resources (use in this order):

- **Reference** (use heavily): Resume Template for AI Roles (in this document) → [see Resume Template section](#)
- **Read** (20 min): GitHub profile README guide → docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme
- **Browse** (20 min for inspiration): Awesome GitHub profile READMEs → github.com/abhisheknaidu/awesome-github-profile-readme
- **Reference** (as needed): Shields.io for badges → shields.io

Step-by-step tasks:

1. Open the Resume Template section in this document. Copy it into a new Google Doc.
2. Fill in your details. For the projects section, write each as STAR: Situation, Task, Action, Result. Use METRICS where possible (e.g., 'processed 100+ PDFs', '95% accuracy on test set', 'deployed at X URL with Y monthly users').
3. Include 4 of your projects: chatbot OR content studio (pick the better one), document reader OR resume reviewer (pick the better one), chat-with-pdf OR chat-with-youtube (pick the better one), and your capstone.
4. Each project bullet should be 2 lines max. Lead with the impactful word.
5. Skills section: list technologies you ACTUALLY used. Don't pad with things you read about. Order: Python, LLM APIs (Claude, Gemini), RAG, ChromaDB, Streamlit, LangChain, Git, [your other languages from coursework].
6. Export as PDF named 'Jathin_Resume_AI.pdf'. Get feedback from at least 2 people before considering done.
7. Now polish GitHub profile. Create a new repo named EXACTLY your username (e.g., 'jathinm'). Add a README.md to it.
8. GitHub profile README content: hero header with name + tagline, 'I just completed a 30-day AI building sprint. Here are the projects', a table or list of your 8+ projects with links, technologies you

know (using badges from shields.io), where to reach you.

9. Pin your top 6 repos on your GitHub profile (click 'Customize your pins'): the 4 from your resume + 2 strong others.
10. Add a meaningful bio to your GitHub profile (Edit profile): 1 sentence + location + LinkedIn link.
11. Take a fresh, professional profile photo if you don't have one (smartphone is fine, good lighting, plain background).
12. Update LinkedIn 'About' section to match the resume positioning

Tools you'll use: Google Docs, GitHub, Shields.io, Camera/smartphone

✓ **Done when:**

- ☐ Resume PDF named correctly and reviewed by at least 2 people
- ☐ GitHub profile README looks polished and lists 8+ projects
- ☐ 6 repos pinned on GitHub profile (your best work, front and center)
- ☐ Professional photo on both GitHub and LinkedIn
- ☐ GitHub bio + LinkedIn About are aligned and tell the same story

DAY 29 · LinkedIn Optimization + Portfolio Site (6 hours)

Focus: Make sure that when a recruiter Googles your name, they find a serious professional.

What you'll learn today:

- LinkedIn SEO (yes, it's a thing)
- Building a portfolio site fast with a template

Resources (use in this order):

- **Read** (20 min): LinkedIn profile optimization checklist (search any current guide) → [google.com/search?q=linkedin+profile+optimization+2026](https://www.google.com/search?q=linkedin+profile+optimization+2026)
- **Use** (browse): Vercel portfolio templates → vercel.com/templates?type=personal-site
- **Reference** (as needed): Free portfolio template (Next.js) → github.com/timlrx/tailwind-nextjs-starter-blog

Step-by-step tasks:

1. LinkedIn audit. Open your profile, go through each section: Photo (professional, recent, clear face), Banner (custom banner — make one in Canva matching your colors), Headline (use keywords: 'AI Engineer | Python | LLMs | RAG | 3rd Year Engineering Student').
2. About section: 200-300 words, written in first person, ending with what you're looking for ('open to AI engineering internships').
3. Experience: add a 'AI Builder' experience entry covering the past 30 days at 'Self-directed AI program'. List 4-5 key projects as bullets.
4. Education: make sure it's accurate, add expected graduation date.
5. Skills section: add 30+ relevant skills. Order matters — first 3 are most visible.
6. Featured section: pin your best LinkedIn posts here (the carousels you made on Days 7, 14, 21, 27).
7. Get 3-5 LinkedIn endorsements for top skills (ask classmates).
8. Now the portfolio site. Go to vercel.com/templates → search 'portfolio' → pick one you like (the Tailwind/Next.js ones are clean).
9. Click 'Deploy' on the template. It creates a copy in your GitHub and deploys to Vercel.
10. Clone the repo locally. Open in Cursor (Cursor is great for editing template sites — just chat with it: 'change the hero to say my name, add a projects section with these 8 projects').
11. Sections you need: Hero (name, tagline, photo, social links), Projects (8 projects with title, description, tech, image, live link, GitHub link), About (200 words), Contact (email + LinkedIn +

GitHub).

12. Use the same project descriptions from your resume. Use screenshots from your project READMEs.
13. Push to GitHub → Vercel auto-deploys. Your URL is something like `jathin.vercel.app`.
14. Optional: buy domain `jathin.dev` or `jathin.tech` (Namecheap is cheapest, ~₹800/year). Point it to Vercel.
15. Add portfolio URL to: LinkedIn (in About + Contact info), GitHub profile, resume, email signature.

Tools you'll use: LinkedIn, Canva, Vercel, Cursor, GitHub

✓ **Done when:**

- ☐ LinkedIn profile is 100% complete and keyword-rich (test: search 'AI engineer' on LinkedIn — can you find yourself?)
- ☐ Portfolio site is live at a clean URL
- ☐ All 8+ projects on the portfolio site with screenshots and working links
- ☐ Portfolio URL is added everywhere (LinkedIn, resume, GitHub, email signature)
- ☐ If someone Googles your name, the first 3 results are: LinkedIn, GitHub, portfolio site

DAY 30 · Mock Interviews + Plan for Next 30 Days (5 hours)

Focus: *Final day. Prepare for the interviews. Plan what comes next so this momentum doesn't die.*

What you'll learn today:

- Common AI/ML interview questions for entry-level roles
- How to talk about your projects effectively in interviews
- What to do AFTER this program ends

Resources (use in this order):

- **Read** (30 min): Common LLM / AI interview questions (search for recent list) → [google.com/search?q=AI+engineer+interview+questions+2026](https://www.google.com/search?q=AI+engineer+interview+questions+2026)
- **Use** (1 hr): Pramp or Interviewing.io for free mock interviews → pramp.com
- **Use** (30 min): Claude.ai as your mock interviewer (paste the prompt below) → claude.ai

Step-by-step tasks:

1. Write a 'Tell me about yourself' answer (60 seconds spoken). Practice it 10 times until it's natural. Hit: who you are, what you've built, what excites you, what you want next.
2. Prepare your 'Walk me through your favorite project' answer. Pick your capstone or resume reviewer. Cover: problem, approach, tech, hardest challenge, what you'd do differently. 3 minutes.
3. Make a 'common questions' doc. Answer in writing (then practice out loud) for these: 'Explain RAG in 60 seconds.' 'Difference between RAG and fine-tuning?' 'When would you use an agent vs a simple LLM call?' 'How does cosine similarity work?' 'What's a vector database and why do we need it?' 'How would you handle a hallucination in production?' 'Why Claude vs GPT for your projects?' 'What's the most important prompt engineering technique?'
4. Use Claude.ai as mock interviewer. Paste this prompt: 'You are interviewing me for an AI Engineer Intern role at a mid-sized startup. Ask me questions one at a time. Wait for my answer. Give brief feedback after each answer. Cover: technical AI knowledge, project deep-dive, behavioral. Total 8 questions. Begin.'
5. Do the full 8-question mock with Claude. Take notes on weak spots.
6. If you have a friend doing similar prep — do a peer mock interview. 20 minutes each side.
7. Now plan the NEXT 30 days post-program. Write 'next-30-days.md' in your ai-30-day-journey repo. Include: (a) Apply to 30 AI roles/internships (list specific companies you're targeting), (b) Pick a deeper specialization (RAG, agents, fine-tuning, evals — pick ONE), (c) Build 2-3 deeper projects in that specialization, (d) Write 4 more technical blog posts, (e) Network goal: connect with 20 AI

engineers on LinkedIn weekly

8. Final Day 30 LinkedIn post: 'I just finished 30 days of building AI. Here's everything I shipped.' Use Appendix B Day 30 template. Include all 8+ projects. Tag the post #BuildInPublic #AIEngineer.
9. Reflection: write 500 words in your hashnode blog. Title: 'I just spent 30 days learning AI as a 3rd year student. Here's what worked, what didn't, and what I'd do differently.' Be honest. This is your most viewed blog post.
10. Email or message me (Prateek) a summary of what you built. I'll share it on my LinkedIn too if it's good. Free distribution.
11. Celebrate. You finished something most people never start.

Tools you'll use: Claude.ai (as mock interviewer), Pramp, Hashnode, LinkedIn

✓ **Done when:**

- ☐ 'Tell me about yourself' practiced 10+ times, sounds natural
- ☐ Common questions doc with written answers to 8+ questions
- ☐ Completed at least one mock interview (with Claude or a peer)
- ☐ next-30-days.md plan written and committed
- ☐ Final Day 30 LinkedIn post published
- ☐ Reflection blog post published on Hashnode
- ☐ Sent me (Prateek) a summary message

Capstone Project Ideas

Pick ONE on Day 24. These are sized for 4 days of focused work and each one will look impressive on a resume. They're ordered roughly by difficulty (1 = most approachable, 7 = most ambitious).

1. AI Study Buddy

Personalized study companion for engineering students. Upload textbook chapters → ask questions → get explanations + practice problems.

Why this is a good capstone: Universal student appeal. You can demo it with your own coursework. Combines RAG + content generation.

Tech stack: Streamlit, ChromaDB, Anthropic API, PDF parsing

What makes it stand out: Auto-generates quiz questions from the indexed material with difficulty levels.

2. AI Interview Prep Coach

Paste a job description → AI generates 20 likely interview questions → you record video answers → AI gives feedback on each.

Why this is a good capstone: Huge use case. Every student needs this. Multimodal (audio/video) makes it stand out.

Tech stack: Streamlit, Anthropic Vision, Browser audio APIs or OpenAI Whisper, Anthropic API

What makes it stand out: Compares your answer to the question and gives a confidence score + 3 specific improvements.

3. Code Review Agent

Paste a GitHub repo URL → agent fetches the code → analyzes each file → produces a structured code review report with issues, suggestions, and praise.

Why this is a good capstone: Developer-focused, looks technical, agent + tool use. Senior engineers will respect this.

Tech stack: Streamlit, GitHub API, Anthropic tool use, Async processing

What makes it stand out: Generates ranked issues by severity + auto-creates a draft PR comment for each issue.

4. AI Career Coach for Students

Resume + LinkedIn profile audit + role recommendations + 30-day action plan, all personalized via AI.

Why this is a good capstone: Highly relatable, share-worthy, you can market it to your own classmates.

Tech stack: Streamlit, Anthropic API, Structured outputs, PDF parsing

What makes it stand out: Generates a tailored 30-day plan with weekly LinkedIn post suggestions for the user.

5. Lecture-to-Notes AI

Record a lecture (or upload audio) → AI transcribes → generates structured notes with key concepts, definitions, examples, and exam-style questions.

Why this is a good capstone: Audio + LLM. Practical for every student. Demo-able with any YouTube lecture.

Tech stack: Streamlit, OpenAI Whisper (open source, runs free locally) OR Gemini audio API, Anthropic API

What makes it stand out: Exports as a Notion page or PDF with table of contents and timestamp links to original audio.

6. Multi-Language Symptom Checker

User describes symptoms in Hindi/Kannada/English → AI asks clarifying questions → returns possible causes + when to see a doctor (with strong disclaimers).

Why this is a good capstone: Multilingual is rare and impressive. Real social impact angle for India.

Tech stack: Streamlit, Anthropic API (multilingual), Structured output, Conversation state

What makes it stand out: Maintains conversation across multiple turns until it has enough info to give a useful answer. Add disclaimers prominently.

7. Smart Job Search Assistant

Specify what kind of role you want → agent searches LinkedIn / Indeed / Naukri → ranks results by fit → drafts personalized cold emails to each company.

Why this is a good capstone: Agent + multi-step + practical. The most ambitious option. If it works, it's resume gold.

Tech stack: Streamlit, Web scraping (BeautifulSoup), Anthropic tool use, Long-running agent loops

What makes it stand out: Actually works end-to-end on real job sites without breaking on edge cases. Hard part is the scraping reliability.

How to pick

- Pick the one you'd actually USE after the program ends. That gives you intrinsic motivation.
- Don't pick #7 unless you're confident — scraping is a debugging nightmare.
- If you're stuck choosing: pick #1 (Study Buddy) or #4 (Career Coach). They're the most universally useful and ship-able.

Resume Template for AI Roles

Use this for Day 28. Copy into a Google Doc. Replace bracketed [] sections with your details. Keep it to one page.

Header

[Your Full Name]

AI Engineer in Training | 3rd Year Computer Science

[city], India · +91-XXXXXXXXXX · [your email] · linkedin.com/in/[your-handle] ·
github.com/[your-handle] · [your-portfolio].vercel.app

Summary

3rd year Computer Science student who recently completed an intensive 30-day AI engineering sprint, shipping 8 production-grade AI applications. Hands-on experience with LLM APIs, Retrieval-Augmented Generation, vector databases, and AI agents. Comfortable taking projects from idea to deployed product. Looking for AI Engineer internship roles where I can build real systems.

AI Projects

AI Resume Reviewer — [github.com/your/repo] | [live-url]

- Built a production web app that analyzes resumes against job descriptions, returning ATS score, missing keywords, and structured improvement suggestions
- Implemented structured JSON output from Claude using XML-tagged prompts and prefill technique
- Deployed on Streamlit Cloud and Hugging Face Spaces; handled X total user sessions in first 7 days
- Tech: Python, Streamlit, Anthropic Claude API, pypdf, structured prompting

Chat-with-PDF — [github.com/your/repo] | [live-url]

- Built a full RAG pipeline: PDF parsing → semantic chunking → vector embedding → retrieval → augmented LLM generation
- Implemented page-number citations and conversation memory on top of stateless RAG retrieval
- Compared raw implementation vs LangChain implementation; documented trade-offs in a public blog post
- Tech: Python, Streamlit, ChromaDB, Anthropic Claude API, pypdf

Research Agent — [github.com/your/repo] | [live-url]

- Built an AI agent that takes a research topic, searches the web, fetches relevant pages, and produces a structured report with citations
- Implemented multi-tool function calling with Claude (search_web, fetch_url, write_section) and a robust agent loop with iteration limits
- Designed live trace UI so users can watch the agent's reasoning step by step
- Tech: Python, Streamlit, Anthropic tool use, DuckDuckGo Search API, BeautifulSoup

[Capstone Project Name] — [github.com/your/repo] | [live-url]

- [One-sentence problem statement]
- [Technical highlight — what was hard, what you solved]
- [Outcome / metric — usage, accuracy, or scope]
- Tech: [your stack]

Technical Skills

Languages: Python (advanced), JavaScript (intermediate), [your other languages from coursework]

AI/ML: LLM APIs (Anthropic Claude, Google Gemini), Prompt Engineering, RAG, Vector Databases (ChromaDB), AI Agents & Tool Use, LangChain (familiar)

Web/Tools: Streamlit, Next.js (familiar), Vercel, Hugging Face Spaces, Git/GitHub, Linux CLI

Other: Technical writing, Building in public, [your domain skills]

Education

[College Name] · Bachelor of Engineering, Computer Science

[Expected graduation: Month Year] · CGPA: [X.XX]

Relevant Coursework: Data Structures & Algorithms, Database Systems, Operating Systems, Computer Networks, [any AI/ML course]

Publications & Talks

- Technical blog at jathin.hashnode.dev — [X] posts on LLMs, RAG, and AI engineering
- [Any college club / hackathon / talks if applicable]

Resume Tips

- Keep it 1 page. If you have less than 2 years experience, 1 page is the rule, not a guideline.
- Lead each bullet with an action verb: Built, Designed, Implemented, Deployed, Optimized, Compared.
- Include METRICS where you can. Even small ones: '8 projects', '500+ LinkedIn engagements', '95% accuracy on test set'.
- Top 3 projects matter most. Order them by impressiveness, not chronologically.
- Save as: Jathin_FirstName_Resume_AI.pdf — recruiters search for it later by your name.

Daily Progress Tracker

Tick off each day as you complete it. Add a one-line note about what you learned or what was hard. This becomes raw material for your Hashnode posts and your 'Tell me about yourself' interview answer.

Day	Done	What you did	One-line note
1	☐	Orientation & Public Commitment	
2	☐	Python Refresher + Git Daily Flow	
3	☐	How LLMs Actually Work	
4	☐	Prompt Engineering Masterclass	
5	☐	Your First API Call	
6	☐	AI Note Summarizer	
7	☐	Week 1 Recap + Blog Launch	
8	☐	Streamlit Crash Course	
9	☐	AI Chatbot with Memory	
10	☐	AI Content Studio	
11	☐	Image AI – Smart Document Reader	
12	☐	AI Resume Reviewer	
13	☐	Production Polish & HF Spaces	
14	☐	Week 2 LinkedIn Showcase	
15	☐	Embeddings & Vector DBs Theory	
16	☐	ChromaDB Hands-On	
17	☐	Chat with PDF – Part 1	
18	☐	Chat with PDF – Part 2	
19	☐	Chat with YouTube	
20	☐	LangChain Comparison	
21	☐	Week 3 Showcase + Reflection	
22	☐	Understanding AI Agents	
23	☐	Research Assistant Agent	
24	☐	Capstone Day 1 – Plan	

Day	Done	What you did	One-line note
25	☐	Capstone Day 2 – Core Build	
26	☐	Capstone Day 3 – UI	
27	☐	Capstone Day 4 – Deploy	
28	☐	Resume & GitHub Polish	
29	☐	LinkedIn + Portfolio Site	
30	☐	Mock Interviews + Next 30 Days	

Appendix A: README Template

Copy this into every project's README.md. Replace [bracketed] sections with your content. A great README is the single highest-leverage thing you can do for a portfolio project.

```
# 🤖 [Project Name]

> One-sentence description that makes someone WANT to click your demo link

[![Live Demo](https://img.shields.io/badge/Live-Demo-success)](https://your-app.streamlit.app)
[![Hugging Face](https://img.shields.io/badge/👉-Spaces-yellow)](https://huggingface.co/spaces/yourname/project)
[![GitHub](https://img.shields.io/badge/GitHub-Repo-blue)](https://github.com/yourname/project)

![Demo GIF](demo.gif)

## ✨ What it does

[2-3 sentences explaining the user-facing value. Not the tech — the value.]

## 🎯 Features

- ✅ [Feature 1 — what user can do]
- ✅ [Feature 2]
- ✅ [Feature 3]
- ✅ [Feature 4]

## 🏗️ Architecture

[Embed architecture diagram image]

User Input → [Step 1] → [Step 2] → [Step 3] → Output

## 🛠️ Tech Stack

- **Language:** Python 3.12
- **AI:** Anthropic Claude (claude-sonnet-4-6)
- **UI:** Streamlit
- **Deployment:** Streamlit Cloud + Hugging Face Spaces
- **Other:** [other libs you used]

## 🚀 Run Locally

```bash
Clone
git clone https://github.com/yourname/project.git
cd project

Setup
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

```
#Add your API key
echo "ANTHROPIC_API_KEY=your-key" > .env
```

```
#Run
streamlit run app.py
``
```

## 📸 Screenshots

[Add 2-4 screenshots showing key moments]

## 🧠 What I Learned

- [Technical insight 1 that you didn't know before]
- [Technical insight 2]
- [Non-technical insight — what was harder than expected]

## 🎯 Future Improvements

- [] [Feature you'd add next]
- [] [Optimization you'd make]
- [] [Stretch goal]

## 🙌 About Me

Built by **Jathin** as part of a 30-day AI engineering sprint.

[LinkedIn](https://linkedin.com/in/your-handle) ·  
[Portfolio](https://your-site.dev)

[GitHub](https://github.com/your-handle) ·

## 📄 License

MIT



## Appendix B: LinkedIn Post Templates

Copy, customize, post. Don't agonize over wording — done > perfect.

### B.1 Day 1 Announcement Post

Starting something new today.

For the next 30 days, I'm going to build AI tools — every single day.

I'm a 3rd year engineering student. AI is everywhere right now but most of us only USE ChatGPT. I want to know how to BUILD with it.

The plan:

- Week 1: LLM basics + first API calls
- Week 2: Deployed AI apps (chatbots, content tools, image AI)
- Week 3: RAG & vector databases
- Week 4: AI agents + capstone project

I'll post here every weekend with what I built.

Day 1 done: GitHub setup, dev environment ready, first repo live.  
Repo: [github.com/\[your-handle\]/ai-30-day-journey](#)

If you're learning AI too — let's connect. Drop a 🙋 in comments.

#BuildInPublic #AIEngineer #100DaysOfCode #IndianTech

### B.2 Weekly Recap Post (use for Days 7, 14, 21)

Week [X] of 30-day AI sprint: done. ✅

What I built this week:

- 🔧 [Project 1] — [one-line description] → [link]
- 🔧 [Project 2] — [one-line description] → [link]
- 🔧 [Project 3] — [one-line description] → [link]

Biggest learning: [one specific thing you didn't know before — be specific]

Hardest moment: [the one bug or concept that took the longest]  
How I fixed it: [what worked]

Up next (Week [X+1]): [theme] — building [what]

All code public on GitHub: [github.com/\[your-handle\]](#)  
Blog post with details: [hashnode-link]

(For anyone learning AI — happy to share resources. DM me.)

#BuildInPublic #AIEngineer #LearningAI #StudentBuilder

### B.3 Project Launch Post (use for capstone on Day 27)

I just launched [Project Name]. 🚀

What it does:

[2 sentences. Lead with user value, not tech.]

Why I built it:

[1-2 sentences. What problem you saw. Make it personal.]

How it works:

→ [Step 1 in plain English]

→ [Step 2]

→ [Step 3]

What surprised me:

[One specific technical thing that turned out harder OR easier than expected]

Tech stack: Python, Streamlit, Claude API, [your other libs]

Try it live: [URL]

Code: [GitHub URL]

60-sec demo: [Loom URL]

If you find a bug or have feedback — comment below. I'll keep iterating.

#BuildInPublic #AIEngineer #ShipIt

### B.4 Day 30 Wrap Post

30 days ago I posted that I would build AI tools every day.

Today is Day 30. Here's everything I shipped:

1. [Project] — [link]

2. [Project] — [link]

3. [Project] — [link]

4. [Project] — [link]

5. [Project] — [link]

6. [Project] — [link]

7. [Project] — [link]

8. [Capstone] — [link]

What I learned that I didn't expect:

→ [Honest insight 1]

→ [Honest insight 2]

→ [Honest insight 3]

What's next:

Applying to AI engineering internships. If your team is hiring — DMs are open.

Reflection blog (the full story, what worked, what didn't): [Hashnode link]

To anyone thinking about doing something similar: start today. Even a small version.

#BuildInPublic #AIEngineer #30DayChallenge

### **LinkedIn Tips That Actually Work**

- Post between 7-10 AM IST on weekdays — highest engagement window for Indian audience
- Hook line matters most. First line decides if people stop scrolling. Be specific, not generic.
- Short paragraphs (1-2 lines). Long blocks of text on LinkedIn = scroll past.
- Use line breaks deliberately. Empty lines create breathing room.
- Drop URLs in FIRST COMMENT, not in the post itself. LinkedIn deprioritizes posts with outgoing links.
- Reply to every comment in the first 4 hours. The algorithm rewards engagement, not just likes.
- Don't fake-tag big names. Tag people you've actually engaged with.



## Appendix C: Code Boilerplate

Copy these into your projects. Each snippet is the minimum viable code for the pattern. Customize from there.

### C.1 read\_env.py — Loading API keys from .env

```
from dotenv import load_dotenv
import os

load_dotenv() #reads .env file in current folder

name = os.getenv('MY_NAME')
api_key = os.getenv('DUMMY_KEY')

print(f'Hello {name}, your key starts with {api_key[:4]}...')
```

### C.2 Prompt library file template (.md)

```
#[Prompt Name]

Purpose: [What this prompt does in one sentence]

Tags: [zero-shot | few-shot | CoT | XML | role]

Prompt

...

[Your actual prompt text here. Use XML tags if applicable.]
...

Example Input

[What you'd send to the model]

Example Output

[What you got back when you tested it]
```

### C.3 hello\_claude.py — First API call to Anthropic

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic() #reads ANTHROPIC_API_KEY from env

response = client.messages.create(
 model='claude-sonnet-4-5-20250929',
 max_tokens=1024,
 messages=[
 {'role': 'user', 'content': 'Explain RAG in one sentence.'}
```

```
]
)

print(response.content[0].text)
```

## C.4 hello\_gemini.py – First API call to Gemini

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

model = genai.GenerativeModel('gemini-2.5-flash')
response = model.generate_content('Explain RAG in one sentence.')

print(response.text)
```

## C.5 ask.py – CLI starter with Click

```
import click
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()

@click.command()
@click.argument('question')
@click.option('--model', default='claude', type=click.Choice(['claude', 'gemini']))
@click.option('--stream', is_flag=True, help='Stream the response')
def ask(question, model, stream):
 if not question.strip():
 click.echo('Please provide a question.')
 return
 if model == 'claude':
 ask_claude(question, stream)
 else:
 ask_gemini(question) #implement separately

def ask_claude(question, stream):
 client = Anthropic()
 if stream:
 with client.messages.stream(
 model='claude-sonnet-4-5-20250929',
 max_tokens=1024,
 messages=[{'role': 'user', 'content': question}]
) as s:
 for text in s.text_stream:
 click.echo(text, nl=False)
 click.echo()
 else:
 response = client.messages.create(
```

```

 model='claude-sonnet-4-5-20250929',
 max_tokens=1024,
 messages=[{'role': 'user', 'content': question}]
)
 click.echo(response.content[0].text)

if __name__ == '__main__':
 ask()

```

## C.6 Streaming response handling

```

Drop-in pattern: prints tokens as they arrive
from anthropic import Anthropic
client = Anthropic()

with client.messages.stream(
 model='claude-sonnet-4-5-20250929',
 max_tokens=1024,
 messages=[{'role': 'user', 'content': 'Tell me a story.'}]
) as stream:
 for text in stream.text_stream:
 print(text, end="", flush=True)
print() # final newline

```

## C.7 Streamlit hello world

```

import streamlit as st

st.title('Hello, Jathin! 🙌')
st.write('This is my first Streamlit app.')

name = st.text_input('What is your name?')
if st.button('Greet me'):
 st.success(f'Welcome, {name}!')

```

## C.8 Streamlit chat with session state

```

import streamlit as st
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

st.title('AI Chatbot')

Initialize message history
if 'messages' not in st.session_state:
 st.session_state.messages = []

Display existing messages
for msg in st.session_state.messages:
 with st.chat_message(msg['role']):

```



```

 st.markdown(msg['content'])

#Get new user input
if user_input := st.chat_input('Ask me anything'):
 st.session_state.messages.append({'role': 'user', 'content': user_input})
 with st.chat_message('user'):
 st.markdown(user_input)

 with st.chat_message('assistant'):
 response = client.messages.create(
 model='claude-sonnet-4-5-20250929',
 max_tokens=1024,
 messages=st.session_state.messages
)
 reply = response.content[0].text
 st.markdown(reply)
 st.session_state.messages.append({'role': 'assistant', 'content': reply})

```

## C.9 Streamlit sidebar with settings

```

import streamlit as st

with st.sidebar:
 st.header('Settings')
 model = st.selectbox('Model', ['Sonnet', 'Opus', 'Haiku'])
 temperature = st.slider('Temperature', 0.0, 1.0, 0.7)
 system_prompt = st.selectbox('System prompt', [
 'Helpful Assistant',
 'Code Reviewer',
 'Study Tutor'
])
if st.button('Clear chat'):
 st.session_state.messages = []
 st.rerun()

```

## Appendix D: Troubleshooting

90% of small issues you'll hit. Check here before you ping me.

**Error:** ModuleNotFoundError: No module named 'X'

**Why:** Your virtual environment isn't activated, or the package isn't installed in this venv.

**Fix:** Check terminal prompt — does it show (venv)? If not: source venv/bin/activate (Mac/Linux) or venv\Scripts\activate (Windows). Then: pip install [the-missing-module].

**Error:** python: command not found OR 'python' is not recognized as a command

**Why:** Python isn't in your PATH, or you're on Mac/Linux where it's called python3.

**Fix:** Try python3 instead. If still failing on Windows, reinstall Python and CHECK the 'Add to PATH' box during install.

**Error:** AuthenticationError / 401 / Invalid API Key

**Why:** API key is wrong, expired, not set, or has extra whitespace/quotes.

**Fix:** Open .env. Make sure format is: ANTHROPIC\_API\_KEY=sk-ant-... (no quotes, no spaces around =). Verify the key in console.anthropic.com still exists and hasn't been deleted. Make sure your script does load\_dotenv() BEFORE creating the client.

**Error:** I accidentally committed .env to GitHub

**Why:** Forgot to add .env to .gitignore before first commit.

**Fix:** DELETE the API key on console.anthropic.com immediately. Generate a new one. Then: add .env to .gitignore, then run: git rm --cached .env, then commit and push. Update your local .env with the new key. The old key is in git history but won't work anymore.

**Error:** Streamlit app runs locally but fails to deploy on Streamlit Cloud

**Why:** Missing requirements.txt, or wrong package versions, or missing secrets.

**Fix:** Make sure requirements.txt is in your repo root and lists every package you import. Make sure you added API keys to Streamlit Cloud → app → Settings → Secrets in this format: ANTHROPIC\_API\_KEY = "sk-ant-..." (with quotes). Re-deploy.

**Error:** Streamlit app shows: 'streamlit secrets file not found'

**Why:** You used st.secrets['KEY'] but didn't set it up.

**Fix:** For local dev: create .streamlit/secrets.toml file with KEY="value" entries. For Streamlit Cloud: add same in the Secrets section of the app's settings. Don't commit secrets.toml to GitHub — add it to .gitignore.

**Error:** git push rejected: 'updates were rejected because the remote contains work'

**Why:** GitHub repo has commits your local doesn't (usually from creating README online).

**Fix:** git pull origin main --rebase, then git push origin main.

**Error:** Permission denied (publickey) when pushing to GitHub

**Why:** SSH key not set up; you're using SSH URL but no key.

**Fix:** Easy fix: switch to HTTPS. git remote set-url origin https://github.com/USER/REPO.git. Use Personal Access Token as password (Settings → Developer Settings → Personal access tokens in GitHub).

**Error:** youtube-transcript-api: 'TranscriptsDisabled' or 'NoTranscriptFound'

**Why:** That specific video doesn't have transcripts available.

**Fix:** Catch the exception and show user a friendly message: 'This video doesn't have a transcript available. Try another video.' Some videos genuinely don't have transcripts — this isn't a bug.

**Error:** ChromaDB collection persists between runs but I want a fresh start

**Why:** You used PersistentClient which saves to disk.

**Fix:** Either: delete the ./chroma\_db folder. OR: client.delete\_collection('name'). OR: use chromadb.Client() (in-memory) instead of PersistentClient for testing.

**Error:** Pypdf: gibberish text extracted from PDF

**Why:** PDF is image-based (scanned) — there's no text layer to extract.

**Fix:** Use OCR instead: pytesseract or Gemini Vision API to extract text from page images. Or just inform the user that scanned PDFs aren't supported.

**Error:** Claude returns text outside of expected JSON format

**Why:** You asked for JSON but didn't use prefill or strict instructions.

**Fix:** Add to your prompt: 'Respond ONLY with valid JSON. Do not include any text before or after the JSON object.' Then use prefill — add { as the start of Claude's response. Parse with json.loads(). Catch JSONDecodeError gracefully.

**Error:** Streamlit app is slow / API calls take forever

**Why:** Could be: slow API, no streaming, no caching, doing work on every rerun.

**Fix:** (1) Use streaming for chat (instant feedback). (2) Use @st.cache\_data for expensive computations that don't change. (3) Move slow setup (loading embeddings model) outside main flow. (4) Show st.spinner() so user knows it's working.

**Error:** Hugging Face Space build fails

**Why:** Often Python version mismatch or missing system dependencies.

**Fix:** In your Space, click 'Files' → click '...' → 'Settings'. Set Python version (3.10 or 3.11 are safest). Check README.md at root has correct YAML config (sdk: streamlit, app\_file: app.py).

**Error:** pip install hangs forever or times out

**Why:** Network issues, or pip is trying to compile packages.

**Fix:** Try: pip install --upgrade pip first. Use a different network. If still failing, install one package at a time to find the culprit. Some packages need build tools (Windows: install Visual Studio Build Tools).



# Appendix E: Glossary

Every term you'll hear in this program (and in AI interviews) explained in one or two lines.

Agent	An LLM that can take actions in the world using tools — search, fetch, write, etc. — and iterates until it solves a goal.
API (Application Programming Interface)	A way for your code to talk to another company's service over the internet.
API Key	A secret string that proves you're allowed to use an API. Treat it like a password — never commit to GitHub.
ATS (Applicant Tracking System)	Software that scans resumes before a human sees them. Most large companies use one.
Chain of Thought (CoT)	A prompting technique where you ask the LLM to show its reasoning step-by-step. Improves accuracy on hard problems.
ChromaDB	An open-source vector database. The easiest to learn on. Runs locally.
Chunk / Chunking	Splitting a long document into smaller pieces (~500 chars typically) so it fits in vector DBs and LLM context.
Context Window	The maximum tokens an LLM can read and respond to in one call. Claude Sonnet 4.6 has 200K tokens.
Cosine Similarity	The math used to measure how similar two embeddings are. Range -1 to 1; higher = more similar.
Embedding	A list of numbers (e.g., 1536 numbers) that represents the meaning of a piece of text. Similar meanings = similar numbers.
Few-Shot Prompting	Giving the LLM 2-5 example input/output pairs before asking it to do the task. Often beats zero-shot.
Fine-Tuning	Training a model further on your specific data. Powerful but expensive — usually RAG is the better choice.
Function Calling / Tool Use	Letting the LLM call functions you define. This is what makes agents possible.
Hallucination	When an LLM confidently makes things up that aren't true. Mitigated by RAG and good prompting.
Hugging Face Spaces	Free hosting for AI apps with 16GB RAM. Good backup deployment to Streamlit Cloud.
JSON Mode / Structured Output	Asking the LLM to return data in a specific format (JSON) so your code can parse it reliably.
LangChain	A Python framework that wraps LLM workflows. Helpful for prototyping, sometimes overkill.
LLM (Large Language Model)	A neural network trained on huge text data. Generates text. Examples: Claude, GPT, Gemini.
Multimodal	Working with multiple input types (text + image + audio). Gemini and Claude both support vision.
Prefill	Anthropic technique — putting words in Claude's mouth to force a

	specific output format. Example: end your prompt with { to force JSON.
<b>Prompt Engineering</b>	The skill of writing inputs to LLMs that get the output you want. Highest-leverage AI skill.
<b>RAG (Retrieval-Augmented Generation)</b>	Pattern: retrieve relevant info from a database → include it in the LLM prompt → get a grounded answer.
<b>REST API</b>	A common style of web API where you send HTTP requests and get JSON responses.
<b>Retrieval</b>	The 'R' in RAG — searching a vector database for relevant chunks given a user question.
<b>SDK (Software Development Kit)</b>	An official Python (or other language) library that wraps an API. Easier than raw HTTP calls.
<b>Semantic Search</b>	Searching by meaning rather than exact keywords. Powered by embeddings.
<b>Streamlit</b>	Python library that makes building web UIs easy. Used in every Week 2-3 project of this program.
<b>System Prompt</b>	Instructions to the LLM that shape its behavior across all messages. Sets persona, rules, format.
<b>Temperature</b>	A setting that controls randomness. 0 = deterministic, 1.0 = creative. Use 0 for facts, 0.7-1.0 for creativity.
<b>Token</b>	A unit of text the LLM processes. Roughly 0.75 words for English. APIs charge by tokens.
<b>Tool Use</b>	Same as function calling — letting the LLM call functions you define to take actions.
<b>Top-p / Nucleus Sampling</b>	Alternative to temperature for controlling randomness. Don't worry about it for now.
<b>Vector Database</b>	A database optimized for storing embeddings and finding nearest neighbors. ChromaDB, Pinecone, Weaviate.
<b>Vision API</b>	An LLM endpoint that accepts images as input. Used in Day 11 (Document Reader).
<b>Zero-Shot Prompting</b>	Asking the LLM to do a task with no examples — just instructions. Often enough for simple tasks.

# Ground Rules & FAQ

## Ground Rules

- Build in public on LinkedIn every weekend. Non-negotiable. This single habit converts to interviews.
- Push to GitHub every single day. Even if it's just a README edit. The green-square graph matters.
- Don't skip days 1-7. They feel basic but they're foundational. Skipping = struggling later.
- Don't fake it. If a day's tasks aren't actually done, mark it incomplete and come back. Untested code on GitHub = embarrassment when a recruiter asks about it.
- Use AI to help you learn, not to do it for you. Cursor/Claude.ai is a tutor, not a ghost-writer. Type out the code yourself even if AI suggested it.
- Take Sundays slightly lighter (4 hours, not 6). Sustainable pace beats burnout heroics.

## FAQ

### Q: I missed a day. Should I double up tomorrow?

A: No. Shift forward by one day. Doubling up burns you out and produces low-quality work. Better to do 31 days at 5 hrs each than 30 days with one 10-hour catastrophe in the middle.

### Q: I'm stuck on something for over an hour. What do I do?

A: In order: (1) Read the error message carefully. (2) Check Appendix D. (3) Ask Claude.ai — paste your code and the error. (4) Google the exact error message. (5) Ping me on WhatsApp/Slack. Don't skip step 1-3 before step 5.

### Q: I don't understand a concept even after the resources.

A: That's normal. Different sources explain things differently. Try: (1) Ask Claude.ai 'Explain X to me like I'm 15.' (2) Watch a different YouTube video on the topic. (3) Skip ahead and come back — sometimes understanding comes from seeing it applied.

### Q: Should I use ChatGPT or Claude?

A: For this program, default to Claude (Anthropic). It's the cleanest API to learn on, has the best documentation, and the prompt engineering patterns transfer to other LLMs. Once you finish the program, you'll easily pick up the other APIs.

### Q: My laptop is slow. Will this still work?

A: Yes. Almost everything runs in the cloud (APIs, Streamlit Cloud, Hugging Face). Your laptop just needs to run VS Code and a browser. Even a 4-year-old laptop with 8GB RAM is fine.

### Q: I can't afford the \$5 Anthropic credit right now.

A: Use Gemini API only (free, generous limits). The patterns are nearly identical. You'll lose a small amount of skill transfer but you can do 80% of the program on Gemini alone. Switch to Claude when you can.

### Q: How do I know if my projects are 'good enough'?



A: Three tests: (1) Can your non-technical friend use it without you explaining? (2) Does the README make someone want to click the live demo? (3) If a recruiter clicked the GitHub link cold, would they think 'I want to talk to this person'? If yes to all three — it's portfolio-grade.

**Q: What if I want to keep going after 30 days?**

A: Day 30's tasks include planning the next 30 days. Common paths: (1) Specialize deeper (pick RAG or agents and go deep). (2) Apply to 30 internships. (3) Contribute to open source AI projects. (4) Build a more ambitious capstone with another person.

**Q: Should I learn ML theory (gradient descent, backprop, transformers from scratch)?**

A: Not for this program. It's a different career path (research). For AI ENGINEER roles in 2026, the skills in this program are what gets you hired. If you find you love it after 30 days, then dive deeper into ML theory.

**Q: How important is the LinkedIn building-in-public part really?**

A: Honestly? It's the single highest-ROI activity in this program. Most 3rd year students have GitHub repos nobody sees. Posting your work on LinkedIn weekly is how you become visible. I've seen students get internship DMs from this alone. Don't skip it.

**Q: I'm scared to post on LinkedIn — feels cringe.**

A: Everyone feels this. The cringe goes away after post 3. Use the templates in Appendix B. The students who push through the cringe get the opportunities. The ones who don't... stay invisible.

**Q: Can I do this program with a friend?**

A: Strongly recommended. Pair up. Same schedule, weekly review calls, debug each other's code, comment on each other's LinkedIn posts. Accountability is the #1 predictor of finishing.

***Go build, Jathin.***

*30 days from now you'll be a different engineer.*

---

— Prateek

Co-founder, Pinnacle Media · @howprateek